Asignatura MATI · Curso 2026–27

El problema del viajante sobre la red de carreteras de España

Construcción del grafo vial, algoritmos exactos, heurísticos y metaheurísticos, y estudio empírico de su complejidad



Raimundo, Pablo, Juan y Jiahao

Noviembre de 2026

Código, datos y web del trabajo: repositorio `MATI_TRABAJO` en GitHub

Resumen

Este trabajo estudia el *problema del viajante* (TSP) sobre la red de carreteras real de la España peninsular. Partimos de un extracto de OpenStreetMap, del que construimos un grafo vial simplificado y fuertemente conexo con 179.594 nodos y 309.250 aristas dirigidas. A partir de él calculamos, para 75 ciudades (las 47 capitales de provincia peninsulares y 28 municipios de más de 100.000 habitantes), tablas de distancias por camino mínimo que dan lugar a 480 versiones del problema (240 grafos de ciudades distintos), según el tipo de carreteras, la medida (kilómetros o tiempo de viaje), el sentido de las vías y el ámbito geográfico.

Implementamos ocho algoritmos que cubren las familias clásicas: tres exactos (fuerza bruta, Held–Karp y DFJ), tres heurísticos o aproximados (vecino más cercano, 2-opt y Christofides) y dos metaheurísticas (recocido simulado y algoritmo genético). Los comparamos en un estudio empírico de 4.706 ejecuciones sobre las 240 versiones y sobre subconjuntos aleatorios de 5 a 75 ciudades, midiendo tiempo, memoria y desviación respecto al óptimo, y contrastamos la complejidad teórica con la observada en notación O.

Los resultados muestran que los exactos de enumeración crecen de forma exponencial (la fuerza bruta se agota en 11 ciudades y Held–Karp, por memoria, en unas 20), mientras que la programación entera (DFJ) resuelve las 75 ciudades al óptimo en unos 4 s; que el 2-opt ofrece la mejor relación entre calidad (1–5 % sobre el óptimo) y tiempo (milisegundos); y que el tipo de red y el sentido de las vías modifican el óptimo tanto como la elección del algoritmo. Completan el trabajo una web interactiva que anima los algoritmos sobre el mapa y una guía de uso para el equipo.

Palabras clave: problema del viajante, OpenStreetMap, grafos, algoritmos exactos, heurísticas, metaheurísticas, complejidad computacional.

Abstract

This work studies the *travelling salesman problem* (TSP) on the real road network of mainland Spain. From an OpenStreetMap extract we build a simplified, strongly connected road graph with 179,594 nodes and 309,250 directed edges. For 75 cities (the 47 mainland provincial capitals plus 28 municipalities with over 100,000 inhabitants) we compute shortest-path distance tables that give rise to 480 problem versions (240 distinct city graphs), depending on road type, cost measure (kilometres or travel time), one-way handling and geographic scope.

We implement eight algorithms covering the classical families: three exact (brute force, Held–Karp and DFJ integer programming), three heuristic or approximation algorithms (nearest neighbour, 2-opt and Christofides) and two metaheuristics (simulated annealing and a genetic algorithm). An empirical study of 4,706 runs over the 240 versions and over random subsets of 5 to 75 cities measures time, memory and deviation from the optimum, and compares theoretical with observed complexity in O notation. Enumeration-based exact methods grow exponentially, whereas integer programming solves all 75 cities to optimality in about 4 s; 2-opt gives the best quality–time trade-off. An interactive web application animating the algorithms on the map and a user guide complete the work.

Índice general

Resumen II

Abstract III

1. Introducción 1
- 1.1. El problema del viajante . . . 1
- 1.2. Motivación: carreteras reales . . . 2
- 1.3. Objetivos . . . 2
- 1.4. Qué se ha hecho . . . 3
- 1.5. Herramientas y reproducibilidad . . . 4
- 1.6. Estructura de la memoria . . . 4

2. Datos y construcción del grafo vial 5
- 2.1. Fuente de los datos . . . 5
- 2.2. Un primer enfoque descartado: la API Overpass . . . 5
- 2.3. Filtrado y construcción del grafo . . . 6
  - 2.3.1. Simplificación . . . 6
  - 2.3.2. Conectividad . . . 6
- 2.4. Atributos del grafo . . . 6
  - 2.4.1. Formato de archivo . . . 7
- 2.5. Descripción del grafo obtenido . . . 7
- 2.6. Memoria y rendimiento: reescribir la simplificación . . . 8
- 2.7. Por qué incluir las carreteras secundarias . . . 8
- 2.8. Limitaciones . . . 9

3. Ciudades y versiones del grafo 10
- 3.1. Las ciudades . . . 10
- 3.2. De las ciudades a la red . . . 11
- 3.3. Tablas de distancias . . . 11
- 3.4. Las versiones del problema . . . 12
  - 3.4.1. Cuántas versiones hay . . . 12
  - 3.4.2. Las cuatro versiones de carreteras . . . 12
- 3.5. Decisiones de diseño . . . 13
- 3.6. Comprobaciones . . . 13

4. Los algoritmos 14
- 4.1. Criterios de elección . . . 14
- 4.2. Notación y contrato común . . . 14
- 4.3. Fuerza bruta . . . 15
- 4.4. Held–Karp (programación dinámica) . . . 16

4.5. DFJ: programación entera con restricciones de subciclo . . . 17
4.6. Vecino más cercano . . . 18
4.7. 2-opt . . . 18
4.8. Christofides . . . 19
4.9. Recocido simulado . . . 20
4.10. Algoritmo genético . . . 21
4.11. Quién hizo qué . . . 22

5. Metodología experimental **23**
5.1. Qué se mide . . . 23
5.2. Instancias: dos estudios complementarios . . . 23
5.3. Qué algoritmo se puede aplicar a qué versión . . . 24
5.4. Medir el tiempo: paralelo y en serie . . . 24
5.5. Algoritmos con azar y otros controles . . . 25
5.6. Validación cruzada y control de calidad . . . 25
5.7. Amenazas a la validez . . . 25

6. Resultados **26**
6.1. Validación: los algoritmos exactos coinciden . . . 26
6.2. Complejidad: tiempo y memoria frente al tamaño . . . 26
6.2.1. Complejidad en notación *O*: teoría frente a observación . . . 27
6.3. De enumerar a ser inteligente: la mejora entre algoritmos exactos . . . 28
6.4. Calidad: cuánto se aleja del óptimo lo que no lo garantiza . . . 28
6.4.1. Variabilidad de los algoritmos con azar . . . 29
6.5. Efecto de la versión del grafo . . . 30
6.6. Calidad frente a tiempo: ¿qué conviene usar? . . . 31
6.7. Cuánto distorsiona medir en paralelo . . . 32

7. Conclusiones **33**
7.1. Respuestas a las preguntas del estudio . . . 33
7.2. Otras conclusiones . . . 34
7.3. Limitaciones . . . 35
7.4. Trabajo futuro . . . 35

8. La web interactiva **36**
8.1. Propósito . . . 36
8.2. Qué ofrece . . . 36
8.3. Cómo está hecha . . . 37
8.4. Cómo sabemos que es correcta . . . 38
8.5. Publicación . . . 39

9. Organización del trabajo, uso de IA y lecciones aprendidas **40**
9.1. Reparto del trabajo . . . 40
9.2. Flujo de trabajo con GitHub . . . 40
9.3. Uso de herramientas de inteligencia artificial . . . 41
9.4. Problemas encontrados y lecciones aprendidas . . . 41

9.5. Reproducibilidad 42

A. Reproducir el trabajo **43**

B. Estructura del repositorio **44**

C. Fuentes y licencias **45**

Bibliografía **46**

Capítulo 1

Introducción

1.1 El problema del viajante

El *problema del viajante* (en inglés, *travelling salesman problem*, TSP) es uno de los problemas clásicos de la optimización combinatoria. Un viajante debe visitar un conjunto de ciudades, pasando por cada una exactamente una vez, y volver a la ciudad de partida recorriendo la menor distancia posible. Aunque el enunciado es sencillo, encontrar la mejor ruta es difícil: con n ciudades hay $(n-1)!$ recorridos posibles, un número que crece más deprisa que cualquier exponencial.

Formalmente, sea $V = \{0, 1, \dots, n-1\}$ el conjunto de ciudades y $c(i, j) \geq 0$ el coste de ir de la ciudad i a la ciudad j. Una ruta es una permutación π de V y su coste, incluida la vuelta al inicio, es

$$C(\pi) \;=\; \sum_{k=0}^{n-2} c\big(\pi_k, \pi_{k+1}\big) \;+\; c\big(\pi_{n-1}, \pi_0\big). \tag{1.1}$$

El problema consiste en encontrar la permutación π^* que minimiza $C(\pi)$. Si $c(i, j) = c(j, i)$ para todo par de ciudades el problema es *simétrico*; en caso contrario es *asimétrico*. Si además se cumple la *desigualdad triangular*,

$$c(i, k) \;\leq\; c(i, j) + c(j, k) \qquad \text{para todo } i, j, k \in V, \tag{1.2}$$

decimos que el problema es *métrico*; esta propiedad es necesaria para algunas garantías de aproximación, como la del algoritmo de Christofides (sección 4.8).

El TSP es un problema NP-difícil [17, 10]: no se conoce ningún algoritmo que lo resuelva en tiempo polinómico en el caso general, y se cree que no existe. Eso no significa que sea imposible resolverlo en la práctica. Los algoritmos exactos modernos, basados en programación entera, han resuelto instancias de decenas de miles de ciudades [1], y las heurísticas obtienen soluciones muy buenas en tiempos pequeños. La cuestión interesante es *cuánto cuesta* cada enfoque y *qué se sacrifica* a cambio de rapidez, que es lo que estudia este trabajo.

1.2 Motivación: carreteras reales

La mayoría de los estudios sobre el TSP usan instancias sintéticas (puntos en el plano con distancia euclídea) o colecciones clásicas. Aquí lo planteamos sobre un caso real: *las ciudades de España y las carreteras que las unen*. Esto añade tres ingredientes que no aparecen en el plano:

- **Las distancias son por carretera**, es decir, caminos mínimos sobre un grafo vial con cientos de miles de nodos, no distancias en línea recta.
- **Hay vías de sentido único** (por ejemplo, las dos calzadas de una autovía son aristas distintas), de modo que ir de A a B puede costar distinto que de B a A. El problema es entonces asimétrico, y la elección entre un modelo dirigido y otro no dirigido pasa a ser parte del estudio.
- **Se puede medir en kilómetros o en tiempo**, y la ruta más corta no tiene por qué ser la más rápida.

Estos ingredientes permiten, además, plantear preguntas que un benchmark sintético no permite: ¿cómo cambia el recorrido óptimo si solo se pueden usar autovías? ¿Cuánto cuesta ignorar los sentidos únicos? ¿Qué algoritmos se pueden aplicar a qué versión del problema, y por qué?

1.3 Objetivos

El objetivo general es **medir cómo mejoran en complejidad y en tiempo los distintos algoritmos del viajante al buscar una solución óptima o completa, y qué se pierde en calidad cuando se renuncia al óptimo**. Lo desglosamos en cinco preguntas, que guían todo el estudio:

P1. ¿Qué algoritmo puede usarse con qué versión del grafo, y por qué?

P2. ¿Cómo crecen el tiempo y la memoria con el número de ciudades? ¿Coincide con la complejidad teórica?

P3. ¿Cuánto se aleja de la ruta óptima cada algoritmo que no garantiza el óptimo?

P4. ¿Cómo cambian los resultados con el tipo de carreteras, el peso (kilómetros o tiempo) y el sentido de las vías?

P5. ¿Qué algoritmo conviene en cada caso, atendiendo a calidad y tiempo?

Para responderlas hay que resolver antes problemas previos de ingeniería de datos: obtener un grafo vial fiable, elegir y justificar un conjunto de ciudades, y definir un conjunto de *versiones* del problema bien especificadas y reproducibles.

1.4 Qué se ha hecho

El trabajo se organiza en cinco partes encadenadas (figura 1.1), a las que se suman una web interactiva y esta memoria:

Parte 1. **Construcción del grafo.** Descarga de un extracto de OpenStreetMap, filtrado, simplificación y asignación de longitudes y tiempos (capítulo 2).

Parte 2. **Ciudades y versiones del grafo.** Selección de 75 ciudades y cálculo de tablas de distancias que originan 480 versiones del problema (capítulo 3).

Parte 3. **Algoritmos.** Implementación de ocho algoritmos, tres exactos, tres heurísticos y dos metaheurísticos (capítulo 4).

Parte 4. **Estudio empírico.** Ejecución sistemática de los algoritmos y análisis de tiempo, memoria y calidad (capítulos 5 y 6).

Parte 5. **Conclusiones.** Respuesta a las preguntas planteadas y recomendaciones de uso (capítulo 7).

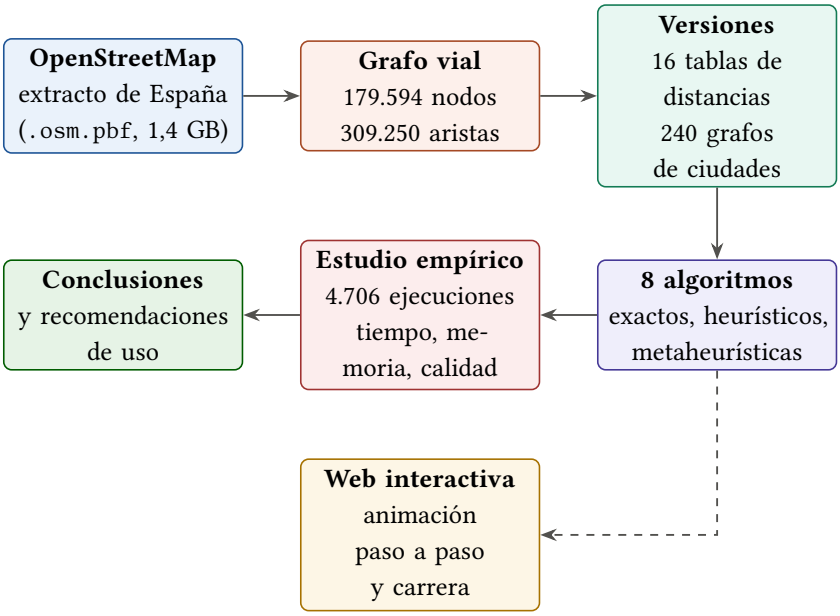


Figura 1.1. Esquema del trabajo: de los datos de OpenStreetMap a las conclusiones. La web reutiliza los datos y los algoritmos.

1.5 Herramientas y reproducibilidad

Todo el trabajo está escrito en Python 3 y se entrega como un *notebook* (`trabajo_tsp.ipynb`) más tres scripts de apoyo (los *anexos*): el Anexo A descarga el extracto, el Anexo B construye el grafo y el Anexo C ejecuta el estudio empírico en paralelo. Usamos NetworkX [11] para los grafos, NumPy [13] y SciPy [20] para el cálculo numérico, PuLP con el *solver* CBC para la programación entera de DFJ y OSMnx [3] únicamente en la construcción del grafo. El código, los datos intermedios y los resultados se versionan en GitHub, y cada resultado de esta memoria se puede regenerar con los comandos del apéndice A.

1.6 Estructura de la memoria

El capítulo 2 describe los datos y la construcción del grafo vial. El capítulo 3 presenta las ciudades y las versiones del problema. El capítulo 4 explica los ocho algoritmos y justifica su elección. El capítulo 5 detalla la metodología del estudio y el capítulo 6 presenta sus resultados. El capítulo 7 recoge las conclusiones. El capítulo 8 describe la web interactiva y el capítulo 9 la organización del trabajo en equipo, el uso de herramientas de inteligencia artificial y los problemas encontrados. Los apéndices reúnen las instrucciones para reproducir el trabajo y material complementario.

Capítulo 2

Datos y construcción del grafo vial

Este capítulo explica de dónde salen las carreteras, cómo se convierten en un grafo utilizable por los algoritmos y qué decisiones de diseño se tomaron en el camino, incluidas las que no funcionaron.

2.1 Fuente de los datos

Las carreteras proceden de *OpenStreetMap* (OSM) [12], una base de datos geográfica colaborativa publicada bajo la licencia ODbL (*Open Database License*), que obliga a citar la fuente como «© OpenStreetMap contributors». No es una fuente oficial: la calidad de los datos depende de la comunidad de mapeadores, lo que implica que algunas etiquetas (por ejemplo, la velocidad máxima de un tramo) están incompletas. Aceptamos esta limitación porque OSM es la única fuente abierta que describe toda la red con la topología necesaria (tramos, sentidos de circulación y jerarquía de las vías).

Usamos el extracto de España de *Geofabrik* (`spain-latest.osm.pbf`, 1,4 GB, última modificación el 19 de octubre de 2026). Para las ciudades empleamos datos oficiales del INE (capítulo 3).

2.2 Un primer enfoque descartado: la API Overpass

El primer enfoque fue descargar la red directamente con OSMnx [3] a través de la API pública *Overpass*, dividiendo España en 324 consultas. En unas once horas solo se completaron 19 consultas (el 6 %), por errores `ConnectTimeout` y `502 Bad Gateway` del servidor público, que estaba saturado; además, una de sus dos direcciones IP no aceptaba conexiones y OSMnx fija una sola IP por servidor. Estimamos que completar la descarga llevaría días y no sería reproducible. Se conserva el registro de aquel intento como evidencia (`docs/intento_overpass/`), pero se descartó.

Decisión. Sustituir Overpass por un único archivo descargable y reanudable (el extracto de Geofabrik) y filtrarlo en local. La descarga completa tarda unos siete minutos y puede repetirse.

2.3 Filtrado y construcción del grafo

El extracto de 1,4 GB contiene todos los elementos de OSM de España. Nos interesa solo la red de carreteras de coche, así que conservamos las vías cuya etiqueta `highway` es `motorway` (autopista y autovía), `trunk` (vía rápida), `primary` (red principal) o `secondary` (red secundaria), junto con sus enlaces (`*_link`). El filtrado se hace con la biblioteca `pyosmium` en dos pasadas: la primera selecciona las vías que cumplen el filtro; la segunda recupera las coordenadas de los nodos que esas vías usan.

El grafo resultante es un *multigrafo dirigido*: cada tramo de carretera de doble sentido se representa con dos aristas, una en cada dirección, y una autovía con calzadas separadas son dos aristas distintas. Se admite más de una arista entre el mismo par de nodos (carreteras paralelas), que se distinguen por una clave.

2.3.1 Simplificación

Un grafo con todos los puntos del trazado tendría millones de nodos. Se *simplifica*: solo se conservan como nodos las intersecciones y los extremos de carretera, y cada arista representa un tramo completo entre dos de ellos, con su longitud total. Seguimos las mismas reglas que OSMnx: un nodo se conserva si tiene bucle sobre sí mismo, si no tiene aristas de entrada o de salida, o si no es un simple punto intermedio (un nodo con exactamente dos vecinos y grado 2 o 4 se considera intermedio y se fusiona). La longitud de un tramo es la suma de los segmentos originales, medidos con la distancia ortodrómica (*great-circle*) entre coordenadas.

2.3.2 Conectividad

Para que exista un camino entre cualquier par de nodos, se conserva la mayor componente *fuertemente conexa* del grafo, es decir, aquella donde se puede ir de cualquier nodo a cualquier otro respetando los sentidos de circulación. Esto elimina también los fragmentos aislados (islas, tramos desconectados, extremos de frontera). El grafo base final es fuertemente conexo y cubre únicamente la **España peninsular**: Baleares, Canarias, Ceuta y Melilla no están.

2.4 Atributos del grafo

Cada arista guarda tres valores numéricos y varios descriptivos. La velocidad se toma de la etiqueta `maxspeed` de OSM cuando existe; si falta, se imputa la media de los tramos de su mismo tipo de vía. Con la longitud L en metros y la velocidad v en km/h, el tiempo de viaje en segundos es

$$t = \frac{L}{v/3{,}6}. \tag{2.1}$$

Por ejemplo, un tramo de la AP-8 de 5.895,57 m a 100 km/h tiene un tiempo de 212,24 s. La tabla 2.1 resume los atributos.

Los nodos guardan su longitud (x), su latitud (y) y el número de calles que confluyen (`street_count`). OSM no nombra las intersecciones, por lo que *los nodos no tienen nombre*: los nombres de ciudad se asocian a un nodo en el capítulo 3.

Para poder guardar el grafo en un formato estándar se eliminaron dos atributos de OSM: `osmid` (listas de identificadores, inútiles para trabajar y no admitidas por el formato) y `maxspeed` (con listas y valores no numéricos como `ES:urban` o `50 mph`, cuya información ya está resuelta en `speed_kph`). Los atributos que OSM da como lista (`ref`, `name`) se unieron en un único texto.

Tabla 2.1. Atributos de las aristas del grafo base.

Atributo	Tipo	Presencia	Significado
`length`	decimal	100 %	Longitud del tramo en metros (peso para distancias).
`travel_time`	decimal	100 %	Tiempo de viaje en segundos, según la ecuación (2.1) (peso para tiempos).
`speed_kph`	decimal	100 %	Velocidad usada (km/h): la máxima de OSM o la media de su tipo de vía.
`highway`	texto	100 %	Tipo de vía (`motorway`, `trunk`, `primary`, `secondary` o sus enlaces).
`oneway`	booleano	100 %	`True` si es de sentido único (227.446 aristas) o `False` si es de doble sentido (81.804).
`ref`	texto	90,3 % de los km	Código de la carretera (`AP-7`, `A-4`, `N-340`…).
`name`	texto	72,0 % de los km	Nombre en lenguaje natural («Autovía del Mediterráneo»).

2.4.1 Formato de archivo

El grafo se guarda en *GraphML*, un formato estándar de grafos basado en XML, con `nx.write_graphml` de NetworkX. Este formato declara el tipo real de cada atributo, de modo que al leerlo con `nx.read_graphml` los números llegan como números y los booleanos como booleanos, y el *notebook* solo necesita NetworkX, como las prácticas de la asignatura. El archivo ocupa 108 MB y se carga en unos 20 segundos.

2.5 Descripción del grafo obtenido

El grafo base tiene **179.594 nodos** y **309.250 aristas dirigidas** (unos 183.000 km contando cada sentido). Hay 3.349 pares de nodos unidos por más de una arista. La tabla 2.2 reparte las aristas por tipo de vía y la tabla 2.3 clasifica los nodos según el número de calles que confluyen.

Tabla 2.2. Aristas y kilómetros por tipo de vía.

Tipo (`highway`)	Qué es	Aristas	Km
`motorway`	autopista y autovía	17.618	31.485
`trunk`	vía rápida o carretera desdoblada	52.029	31.438
`primary`	carretera de la red principal	75.467	40.454
`secondary`	carretera de la red secundaria	100.495	69.150
`*_link`	enlaces y accesos de los cuatro anteriores	63.641	10.233
Total		**309.250**	**182.760**

Tabla 2.3. Tipos de nodo según el número de calles que confluyen (`street_count`).

Calles	Significado típico	Nodos
1	extremos (fin de carretera o corte en la frontera)	1.058
2	empalmes entre dos vías distintas de OSM	4.612
3	bifurcaciones o incorporaciones (entradas y salidas de autovía)	161.516
4	cruces de cuatro ramas	10.332
5 o más	nudos complejos	2.076

2.6 Memoria y rendimiento: reescribir la simplificación

El grafo con carreteras secundarias no cabía en memoria con el flujo habitual de OSMnx (`graph_from_*`): se estimaban unos 12 GB y el proceso fue finalizado por el sistema operativo por falta de memoria. En lugar de eso, el Anexo B reimplementa con *arrays* de NumPy las reglas de simplificación de OSMnx, sin construir un grafo gigante de NetworkX intermedio. El pico de memoria medido es de unos 2,2 GB y la construcción completa tarda unos cuatro minutos.

Para asegurarnos de que la reimplementación es fiel, la comparamos con el grafo *ligero* (solo `motorway`, `trunk`, `primary` y sus enlaces) que ya habíamos construido con el método anterior: los dos tienen los mismos 108.050 nodos (con los mismos identificadores), las mismas 181.006 aristas y las mismas longitudes (110.128,4 km). Solo cambian ligeramente las velocidades imputadas (diferencia media de 0,05 km/h; 136 aristas difieren más de 1 km/h), porque el método anterior elegía de forma arbitraria el tipo de vía de las aristas mixtas y el nuevo usa el de mayor jerarquía.

2.7 Por qué incluir las carreteras secundarias

El grafo base incluye carreteras secundarias; el ligero no. Con solo autopistas, vías rápidas y red principal, una red *sin autovías* queda llena de agujeros: Almería quedaba a 142 km de la red, y la ruta Madrid–Sevilla medía 761 km en lugar de unos 510. Con las secundarias el resultado es utilizable, y además las ciudades vecinas quedan mucho más cerca entre sí (en la red sin secundarias, Terrassa–Sabadell salía un 193 % más lejos). Las versiones más pequeñas del grafo se obtienen después filtrando aristas del grafo base, así que mantener una sola red «maestra» no supone un problema de cómputo.

2.8 Limitaciones

- **Sin geometría.** Al estar simplificadas, las aristas no guardan su forma: dibujadas, son líneas rectas entre intersecciones. Las distancias no se ven afectadas, pero los mapas son aproximados: las rutas dibujadas en la web (capítulo 8) siguen la sucesión de intersecciones del camino mínimo, no la forma exacta de cada curva.
- **Tiempos de coche idealizados.** `travel_time` no incluye tráfico, peajes ni semáforos, y muchas velocidades están imputadas.
- **Longitud geométrica.** `length` es la longitud del trazado en OSM, no el kilometraje oficial.
- **Datos colaborativos.** Pueden contener errores o carencias locales.

Capítulo 3

Ciudades y versiones del grafo

Los algoritmos del viajante no trabajan sobre el grafo vial de 180.000 nodos, sino sobre un grafo mucho más pequeño cuyos nodos son las *ciudades* y cuyas aristas son las distancias entre ellas. Este capítulo explica qué ciudades se eligen, cómo se calculan esas distancias y cómo se obtienen, a partir de una única red, las 480 versiones del problema que se estudian.

3.1 Las ciudades

El conjunto de ciudades son **75 municipios de la España peninsular**: las 47 capitales de provincia peninsulares (50 provincias menos las capitales insulares, Palma, Las Palmas de Gran Canaria y Santa Cruz de Tenerife) y los 28 municipios peninsulares de más de 100.000 habitantes que no son capitales (de los 62 que hay en la península, 34 ya lo son). Se excluyen los municipios insulares de más de 100.000 habitantes (La Laguna y Telde) porque la red vial peninsular no los conecta. De este modo se dispone de dos conjuntos anidados, `capitales` (47 ciudades) y `capitales_y_100k` (75 ciudades), que permiten estudiar cómo crece el problema.

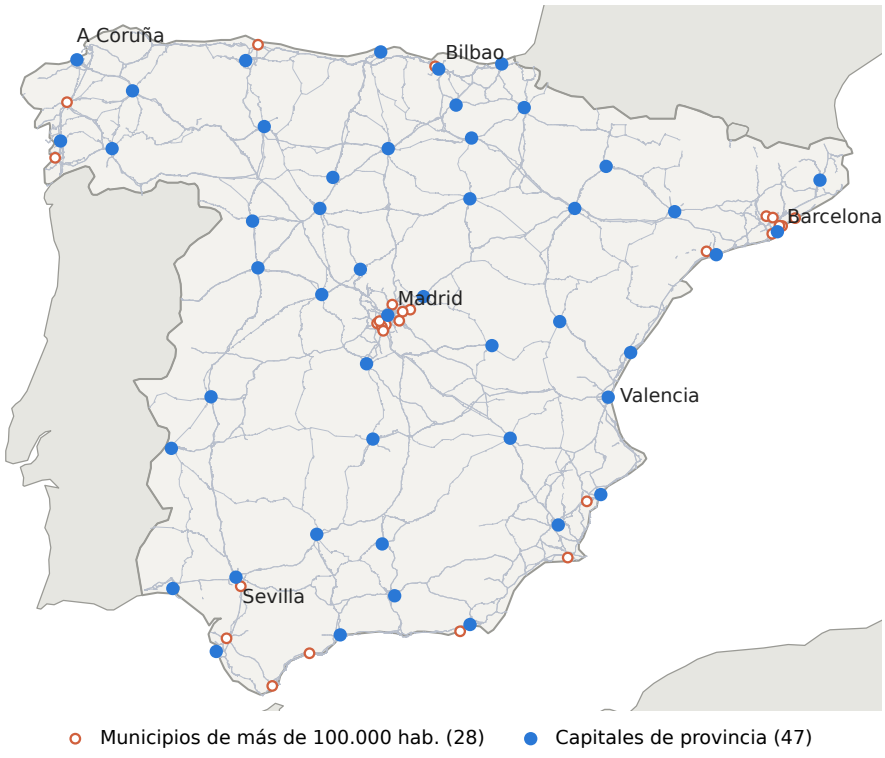


Figura 3.1. Las 75 ciudades del estudio sobre la red de autovías y vías rápidas (fondo gris). Los círculos azules son las 47 capitales de provincia; los círculos naranjas, los 28 municipios de más de 100.000 habitantes que no son capitales.

Fuentes. La lista de capitales y la población proceden del INE (*Cifras oficiales de población de los municipios españoles: Revisión del Padrón Municipal*, datos a 1 de enero de 2025, publicados el 11 de diciembre de 2025 en el BOE-A-2025-25362). La comunidad autónoma de cada ciudad se asignó con las tablas por provincia del INE. Las **coordenadas** proceden de los nodos `place=city` o `place=town` de OpenStreetMap, el centro de cada núcleo urbano; no son una fuente oficial. Una primera geocodificación con Nominatim devolvía la provincia en lugar del municipio para algunas capitales (Guadalajara, Palencia…) y el centro del término municipal en otras (Almería quedaba 14 km desplazada), por lo que se sustituyó por los nodos del propio extracto.

3.2 De las ciudades a la red

Cada ciudad se asocia al *nodo de carretera más cercano*, mediante una búsqueda en un árbol k-d (`cKDTree` de SciPy) sobre coordenadas convertidas a kilómetros. Si ese nodo está a más de $X = 10$ km, la ciudad no está realmente conectada a esa versión de la red y se deja fuera de ella, registrando su distancia. Esto solo ocurre en una versión (sección 3.4.2). La distancia entre la ciudad y su nodo *no* se suma a las distancias: es una simplificación del acceso que afecta por igual a todos los algoritmos.

3.3 Tablas de distancias

Para dos ciudades a y b con nodos asociados u_a y u_b, la distancia entre ellas es la del camino mínimo en el grafo vial:

$$D(a, b) \;=\; \min_{P\colon u_a \to u_b} \sum_{e \in P} w(e), \tag{3.1}$$

donde P recorre las aristas respetando los sentidos y w es la longitud (`length`, que se convierte a kilómetros) o el tiempo de viaje (`travel_time`). Se calcula con el algoritmo de Dijkstra [8] (implementación de SciPy) desde cada una de las 75 ciudades. Como los sentidos únicos hacen que $D(a, b) \neq D(b, a)$, la tabla es asimétrica: por ejemplo, de Madrid a Sevilla hay 502 km y de Sevilla a Madrid, 517 km. Para el caso sin sentidos únicos se ignoran las direcciones y cada tramo cuesta lo que la más barata de sus dos direcciones.

Los caminos mínimos cumplen la desigualdad triangular (1.2) por construcción, de modo que todas las versiones del problema son métricas (las no dirigidas, además, simétricas). Esto es lo que permite aplicar Christofides con garantía a las versiones no dirigidas.

3.4 Las versiones del problema

Una *versión* se define con cinco elecciones (tabla 3.1). Cada versión se carga con una función `cargar_version(…)` que devuelve un grafo de NetworkX cuyos nodos son las ciudades y cuyo peso `peso` es la distancia.

Tabla 3.1. Parámetros que definen una versión del problema.

Parámetro	Valores
`aristas`	`completo` (8 tipos de vía), `autovias_principales` (sin secundarias), `solo_autovias` y `sin_autovias`.
`nodos`	`capitales` (47 ciudades) o `capitales_y_100k` (75 ciudades).
`ambito`	`espana` o una de las 15 comunidades autónomas peninsulares.
`peso`	`km` (kilómetros) o `tiempo` (segundos, el registrado en el grafo).
`general`	`multigrafo_dirigido`, `grafo_dirigido`, `multigrafo_no_dirigido` o `grafo_simple_no_dirigido`.

3.4.1 Cuántas versiones hay

Combinando 4 redes, 2 pesos y dirigido o no dirigido se obtienen **16 tablas** de 75×75 (en total, 0,9 MB). Cada versión es un *trozo* de una de esas tablas, las filas y columnas de las ciudades que correspondan. Guardar 480 grafos de carreteras ocuparía decenas de gigabytes; así, en cambio, quienes programan los algoritmos no necesitan el grafo grande.

El catálogo contiene **480 versiones**, que corresponden a **240 grafos de ciudades distintos**, porque entre dos nodos el camino mínimo solo usa la arista más barata, de modo que el multigrafo y el grafo simple dan exactamente la misma tabla. De los 240 grafos, 32 son de toda España (44 a 75 ciudades) y 208 son regionales (4 a 13 ciudades). Las versiones regionales usan *toda la red* (el mejor camino entre dos ciudades de una comunidad puede salir de ella) y requieren un mínimo de $N_{\text{mín}} = 4$ ciudades, lo que da 5 conjuntos de capitales (Andalucía, Castilla y León, Castilla-La Mancha, Cataluña y Galicia) y 8 de capitales más municipios de más de 100.000 habitantes (los anteriores más Madrid, Comunidad Valenciana y País Vasco). Los tamaños regionales son 4, 5, 6, 8, 9, 11 y 13 ciudades, y son los que permiten aplicar los algoritmos exactos de enumeración a datos reales.

3.4.2 Las cuatro versiones de carreteras

Las cuatro redes se obtienen filtrando aristas del grafo base y conservando, en cada caso, la mayor componente fuertemente conexa (tabla 3.2).

Tabla 3.2. Lo que queda en cada versión de carreteras.

Red	Nodos	Aristas	Km	Nodos descartados	Ciudades fuera
`completo`	179.594	309.250	182.760	0	ninguna
`autovias_principales`	115.476	190.221	111.121	5.885	ninguna
`solo_autovias`	19.239	24.450	28.921	14.027	3: Salamanca (62,7 km), Badajoz (50,5 km), Cuenca (39,6 km)
`sin_autovias`	146.903	256.301	140.040	8.464	ninguna

`autovias_principales` elimina las carreteras secundarias; `solo_autovias` deja únicamente autopistas y autovías; `sin_autovias` excluye autopistas y autovías (la red «para ir por carreteras convencionales»). En `solo_autovias`, tres ciudades quedan a más de 10 km de la red y se dejan fuera, de modo que esa versión tiene 44 o 72 ciudades en lugar de 47 o 75.

3.5 Decisiones de diseño

- **Bicicleta y a pie no son versiones aparte.** Con velocidad constante, el tiempo es proporcional a la distancia, así que el camino más corto y el recorrido óptimo coinciden con los de kilómetros. El caso «a pie o en bici» es `sin_autovias` en kilómetros; para horas basta dividir (entre 20 en bicicleta, entre 5 a pie). Descartamos esas versiones para no duplicar resultados idénticos.
- **Las ciudades fuera de la red no se «arreglan».** Como ocurre con las islas, el viajante no puede visitar una ciudad desconectada; se registra y se deja fuera. Por eso Castilla y León pasa de 9 a 8 capitales en `solo_autovias`.
- **Modelo no dirigido.** Al ignorar los sentidos únicos, cada tramo vale lo que la más barata de sus dos direcciones. Un atajo por una rampa en contradirección es un efecto pequeño, pero existe.
- **Una sola red maestra.** Todo se deriva del grafo base; así cualquier cambio en los criterios (por ejemplo, el umbral de 10 km) se propaga recalculando las 16 tablas, unos dos minutos.

3.6 Comprobaciones

Sobre las 16 tablas se comprueba que las distancias son finitas y no negativas, que la diagonal es cero, que las tablas no dirigidas son simétricas, que la distancia no dirigida nunca supera a la dirigida y que se cumple la desigualdad triangular (con una tolerancia de 0,01 por el redondeo a tres decimales al guardar). Además, se cargaron sin error las 480 versiones del catálogo.

Capítulo 4

Los algoritmos

4.1 Criterios de elección

Hemos implementado ocho algoritmos de modo que estén representadas las familias clásicas de métodos para el TSP, con cuya comparación se puede responder a las preguntas del estudio:

- **Exactos** (garantizan el óptimo): *fuerza bruta*, que enumera todas las rutas; *Held–Karp*, que reduce la enumeración con programación dinámica; y *DFJ*, que usa programación entera con restricciones de subciclo. Cubren tres estrategias muy distintas para llegar al mismo resultado.
- **Heurísticos y aproximados**: el *vecino más cercano* (constructivo, el más simple), el *2-opt* (mejora local sobre una ruta existente) y *Christofides* (aproximación con garantía demostrada).
- **Metaheurísticas**, que exploran el espacio de soluciones con azar: el *recocido simulado* (inspirado en la física) y el *algoritmo genético* (inspirado en la evolución).

Quedó fuera de la lista el algoritmo de Dijkstra, que a veces se cita en este contexto pero no resuelve el viajante: calcula el camino más corto *entre dos puntos*, no un recorrido que pase por todos. En este trabajo se usa precisamente para construir las tablas de distancias entre ciudades (capítulo 3). Tampoco se incluye un «algoritmo hamiltoniano» aparte, porque el TSP es, precisamente, encontrar el ciclo hamiltoniano de menor coste.

4.2 Notación y contrato común

Todos los algoritmos reciben el grafo de ciudades K, completo, donde $K[a][b]$ es el coste $c(a, b)$ de ir de a a b, y una ciudad de partida opcional. Devuelven una ruta (lista con todas las ciudades, cada una una vez, en el orden en que se visitan) y su coste $C(\pi)$ según la ecuación (1.1), que incluye la vuelta. Esta interfaz común es lo que permite comparar los algoritmos con una misma batería de pruebas. Todos funcionan con grafos dirigidos (asimétricos) y no dirigidos, salvo Christofides, que necesita simetría.

La tabla 4.1 resume las propiedades de los ocho algoritmos. En las secciones siguientes se explica cada uno: la idea, el pseudocódigo, la complejidad en notación O y sus propiedades. Las complejidades se expresan en función del número de ciudades n.

Tabla 4.1. Resumen de los ocho algoritmos. G = generaciones, P = población, q = intentos de mutación.

Algoritmo	Autor	Tipo	Tiempo	Memoria
Fuerza bruta	Pablo	exacto	$O((n-1)! \cdot n)$	$O(n)$
Held–Karp	Raimundo	exacto (prog. dinámica)	$O(n^2 \cdot 2^n)$	$O(n \cdot 2^n)$
DFJ	Jiahao	exacto (prog. entera)	$O(2^n)$ en el peor caso	$O(n^2)$ en la práctica
Vecino más cercano	Pablo	heurístico voraz	$O(n^2)$	$O(n)$
2-opt	Raimundo	mejora local	$O(n^2)$ por vuelta	$O(n^2)$
Christofides	Raimundo	aproximado (1,5)	$O(n^3)$	$O(n^2)$
Recocido simulado	Juan	metaheurística	niveles de temperatura × iteraciones	$O(n^2)$
Genético	Raimundo	metaheurística	$O(G \cdot P \cdot n \cdot q)$	$O(P \cdot n)$

4.3 Fuerza bruta

Idea. Probar todas las rutas posibles y quedarse con la más corta. Como cualquier ciclo se puede escribir empezando por la ciudad de partida, hay $(n-1)!$ rutas distintas que hay que evaluar; cada una cuesta $O(n)$ sumar, de ahí la complejidad $O((n-1)! \cdot n)$. La memoria es $O(n)$ porque solo se guardan la mejor ruta y la permutación en curso.

Algoritmo 1 Fuerza bruta

Entrada: grafo K con n ciudades, ciudad de partida s
Salida: ruta óptima y su coste

```
1 mejor ← ∞
2 para cada ordenación σ de las n − 1 ciudades distintas de s hacer
3     ruta ← [s] + σ  ;  coste ← C(ruta)            incluye la vuelta
4     si coste < mejor entonces
5         mejor ← coste  ;  guardar ruta
6     fin si
7 fin para
8 devolver mejor ruta
```

Propiedades. Exacto y determinista; no necesita distancias simétricas. Su problema es el tiempo: 10 ciudades se resuelven en medio segundo, 11 en unos 20 segundos y cada ciudad más multiplica el tiempo por n, de modo que con 13 serían del orden de 50 minutos. Es útil como referencia exacta para instancias muy pequeñas.

4.4 Held–Karp (programación dinámica)

Idea. Muchas de las $(n-1)!$ rutas comparten tramos: si dos rutas empiezan en la ciudad de partida, pasan por las mismas ciudades (en distinto orden) y terminan en la misma, solo importa cuál de los dos tramos es más corto. Held y Karp [14] (y, de forma independiente, Bellman [2]) aprovecharon esa observación con *programación dinámica*: se resuelve cada subproblema una sola vez y se reutiliza.

Se fija una ciudad de partida 0. Para cada subconjunto S de las demás ciudades y cada $j \in S$, sea $d[S][j]$ el coste del camino más corto que sale de 0, visita exactamente las ciudades de S y termina en j. Entonces

$$d[S][j] \;=\; \min_{i \in S \setminus \{j\}} \Big(d\big[S \setminus \{j\}\big][i] + c(i,j) \Big), \qquad d[\{j\}][j] = c(0,j), \tag{4.1}$$

y el coste óptimo es $\min_j \big(d[V \setminus \{0\}][j] + c(j,0) \big)$. La ruta se reconstruye hacia atrás, recordando para cada estado el i que dio el mínimo.

Algoritmo 2 Held–Karp

Entrada: grafo K, ciudad de partida 0
Salida: ruta óptima y su coste

```
d[{j}][j] ← c(0, j) para cada ciudad j ≠ 0
para k ← 2 hasta n − 1 hacer                    tamaño del subconjunto
    para cada subconjunto S de tamaño k y cada j ∈ S hacer
        d[S][j] ← mín_{i∈S\{j}} ( d[S \ {j}][i] + c(i, j) )
    fin para
fin para
j* ← arg mín_j ( d[V \ {0}][j] + c(j, 0) )
devolver ruta reconstruida hacia atrás desde j*
```

Complejidad. Hay $2^{n-1}(n-1)$ estados y cada uno se calcula probando hasta $n-1$ ciudades anteriores, luego el **tiempo** es $O(n^2 \cdot 2^n)$, frente al $O((n-1)!)$ de la fuerza bruta. La **memoria** es $O(n \cdot 2^n)$: la tabla d ocupa $8\,(n-1)\,2^{n-1}$ bytes (unos 80 MB con 20 ciudades, 170 MB con 21 y más de 700 MB con 22). Es el verdadero límite de este algoritmo, y por eso la implementación se niega a trabajar con más de 21 ciudades.

Implementación. Cada subconjunto se guarda como un entero (una máscara de bits) y los subconjuntos de cada tamaño se procesan a la vez con NumPy. **Propiedades:** exacto, determinista, sirve para el caso dirigido y el no dirigido.

4.5 DFJ: programación entera con restricciones de subciclo

Idea. La formulación de Dantzig, Fulkerson y Johnson [7] plantea el TSP como un problema de programación entera con una variable binaria x_{ij} por arista dirigida (vale 1 si el viajante va de i a j):

$$\begin{aligned} \text{mín} \ & \sum_{i \neq j} c(i,j)\, x_{ij} \\ \text{s.a.} \ & \sum_{j \neq i} x_{ij} = 1 \quad \forall i, \qquad \sum_{i \neq j} x_{ij} = 1 \quad \forall j, \qquad x_{ij} \in \{0,1\}. \end{aligned} \tag{4.2}$$

Las dos primeras familias de restricciones (una salida y una entrada por ciudad) definen el *problema de asignación*, que es más fácil que el TSP pero admite soluciones formadas por *varios circuitos sueltos* (subciclos) en lugar de un único ciclo. Para evitarlo se añade, para cada subconjunto S de ciudades que forma un subciclo, una restricción de conectividad

$$\sum_{i \in S} \sum_{j \notin S} x_{ij} \ \geq \ 1, \tag{4.3}$$

que obliga a que al menos una arista salga del grupo. Como hay un número exponencial de subconjuntos, las restricciones no se añaden todas desde el principio: se resuelve el modelo, se detectan los subciclos de la solución, se añaden sus restricciones y se vuelve a resolver, hasta que la solución sea un único ciclo.

Algoritmo 3 DFJ con eliminación iterativa de subciclos

Entrada: grafo K
Salida: ruta óptima y su coste

```
modelo ← problema de asignación (4.2)
repetir
    x* ← resolver el modelo entero con un solver
    C ← componentes (circuitos) de la solución x*
    si |C| > 1 entonces
        para cada circuito S ∈ C hacer
            añadir al modelo la restricción (4.3)
        fin para
    fin si
hasta que |C| = 1
devolver el ciclo x*
```

Complejidad y propiedades. En el peor caso el tiempo es exponencial, porque resolver cada modelo es NP-difícil y el número de restricciones puede crecer exponencialmente. En la práctica, con los *solvers* modernos, suele bastar con muy pocas iteraciones (en nuestras instancias, entre 2 y 10), y la memoria es del orden de $O(n^2)$ por las variables del modelo. Es exacto, determinista y válido para grafos dirigidos y no dirigidos (en este último caso se convierte el grafo a dirigido, con una arista por sentido y el mismo peso). Usamos PuLP con el *solver* CBC. En el capítulo 6 se verá que este algoritmo resuelve las 75 ciudades al óptimo en unos 4 segundos, muy por encima del límite de los exactos de enumeración.

4.6 Vecino más cercano

Idea. Es la ruta del viajante que no planifica: en cada ciudad, ir a la más cercana que todavía no se haya visitado. Es un algoritmo *voraz*: toma siempre la mejor opción local sin mirar atrás. A cambio de su simplicidad renuncia a la optimalidad: una elección buena ahora puede dejar aislada una ciudad lejana y encarecer el final del recorrido.

Algoritmo 4 Vecino más cercano

Entrada: grafo K, ciudad de partida s
Salida: ruta y su coste

1 $ruta \leftarrow [s]$; $pendientes \leftarrow V \setminus \{s\}$
2 **mientras** $pendientes \neq \emptyset$ **hacer**
3 $a \leftarrow$ última ciudad de $ruta$
4 $b \leftarrow \arg\min_{x \in pendientes} c(a, x)$ *la más cercana*
5 añadir b a $ruta$; quitar b de $pendientes$
6 **fin mientras**
7 **devolver** $ruta$, $C(ruta)$ *la vuelta cierra el ciclo*

Complejidad y propiedades. Hay $n - 1$ pasos y en cada uno se comparan hasta $n - 1$ ciudades, luego $O(n^2)$ en tiempo y $O(n)$ en memoria adicional. Es determinista (los empates se resuelven por orden alfabético), no necesita simetría y no ofrece ninguna garantía de calidad: incluso con distancias métricas, el peor caso puede ser $\Theta(\log n)$ veces el óptimo. Depende de la ciudad de partida. Se usa aquí como solución inicial de otras técnicas (2-opt) y como línea base.

4.7 2-opt

Idea. Es una *búsqueda local* [5]: parte de una ruta cualquiera y la mejora con pequeños cambios mientras sea posible. El cambio de 2-opt es el que haría una persona mirando el mapa: si dos tramos de la ruta se cruzan, se deshacen y se vuelven a unir al revés, lo que elimina el cruce y casi siempre acorta el camino.

Concretamente, se toman dos aristas $(a \to b)$ y $(c \to d)$ de la ruta (con b justo después de a y d justo después de c) y se sustituyen por $(a \to c)$ y $(b \to d)$, invirtiendo el tramo que queda entre medias. El cambio mejora la ruta si

$$\Delta \;=\; c(a,c) + c(b,d) - c(a,b) - c(c,d) \;<\; 0, \tag{4.4}$$

donde, en un grafo *dirigido*, hay que sumar además la diferencia entre recorrer el tramo invertido al revés y hacia delante (porque ir de x a y no cuesta lo mismo que de y a x); la implementación lo calcula en tiempo constante con sumas acumuladas y vale igual para el caso no dirigido, donde esa diferencia es cero.

Complejidad y propiedades. Cada vuelta evalúa todos los pares de tramos (con `numpy`, en forma de matrices $n \times n$), luego cuesta $O(n^2)$ en tiempo y memoria; el número de vueltas es del orden de n en la práctica, así que el total ronda $O(n^3)$, aunque no hay una cota polinómica en el peor caso. Es una heurística: se detiene en el primer *óptimo local*, que puede no ser el óptimo global; sin garantía de calidad útil, pero en la práctica queda a pocos puntos porcentuales del óptimo. Es determinista dada la ruta de partida y funciona con grafos dirigidos y no dirigidos.

Algoritmo 5 2-opt

Entrada: grafo K, ciudad de partida s
Salida: ruta (óptimo local) y su coste

```
ruta ← ruta del vecino más cercano desde s
repetir
    para cada par de tramos (a → b), (c → d) no contiguos de ruta hacer
        calcular Δ                                              ecuación (4.4)
    fin para
    (a, b, c, d) ← el par con menor Δ
    si Δ < 0 entonces
        invertir el tramo de b a c en ruta
    fin si
hasta que ningún cambio mejora la ruta
devolver ruta
```

4.8 Christofides

Idea. Es el algoritmo de aproximación con garantía más conocido para el TSP métrico: la ruta que devuelve nunca cuesta más de 1,5 veces el óptimo [4]. Se construye en cinco pasos:

1. **Árbol de expansión mínimo** (*MST*): une todas las ciudades con el menor coste total y sin ciclos. Su coste es una cota inferior del óptimo: si a un ciclo óptimo se le quita una arista queda un árbol, y no puede costar menos que el mínimo.
2. Se identifican las ciudades de **grado impar** en el árbol (siempre hay un número par de ellas).
3. **Emparejamiento perfecto de coste mínimo** entre ellas, que se calcula con el algoritmo de Edmonds [9] (*blossom*). Su coste es, como mucho, la mitad del óptimo.
4. Árbol y emparejamiento forman un multigrafo en el que todas las ciudades tienen grado par, de modo que existe un **circuito euleriano**, que recorre cada arista una vez.
5. **Atajos**: se recorre el circuito saltándose las ciudades ya visitadas. Por la desigualdad triangular (1.2), ir directo nunca cuesta más que dar el rodeo.

La garantía sale de sumar: $C \le c(\text{MST}) + c(\text{emparejamiento}) \le C^* + \frac{1}{2}C^* = 1{,}5\,C^*$.

Algoritmo 6 Christofides (grafo no dirigido)

Entrada: grafo no dirigido K con distancias métricas
Salida: ruta con coste $\le 1{,}5\,C^*$

```
T ← árbol de expansión mínimo de K
I ← ciudades de grado impar en T
M ← emparejamiento perfecto de coste mínimo entre las ciudades de I
circuito ← circuito euleriano del multigrafo T ∪ M
ruta ← circuito sin repetir ciudades (se salta cada ciudad ya visitada)
devolver ruta
```

Complejidad y propiedades. El árbol cuesta $O(n^2 \log n)$, el emparejamiento *blossom* sobre las ciudades impares $O(n^3)$ en el peor caso y el circuito euleriano $O(n)$; domina el emparejamiento, luego $O(n^3)$ en tiempo y $O(n^2)$ en memoria. **Necesita un grafo no dirigido** (distancias simétricas) y métrico,

condiciones que cumplen las versiones no dirigidas de este trabajo. La implementación rechaza grafos dirigidos con un error explícito. Es determinista en lo que depende de los datos.

4.9 Recocido simulado

Idea. El recocido simulado [18] es una metaheurística inspirada en el proceso metalúrgico de recocido. Una búsqueda local como el 2-opt se detiene en el primer valle (óptimo local) porque cualquier cambio inmediato empeora la ruta. El recocido supera esta barrera aceptando también, con cierta probabilidad, cambios que *empeoran* la ruta. Esa probabilidad depende de una *temperatura* T que va bajando poco a poco: a temperaturas altas se acepta casi todo y se explora; al enfriarse, el sistema se vuelve cada vez más voraz y se asienta en un buen valle. Matemáticamente es el algoritmo de Metropolis–Hastings [19] orientado a la optimización: un cambio que aumenta el coste en $\Delta F > 0$ se acepta con probabilidad

$$P \;=\; e^{-\Delta F/T}. \tag{4.5}$$

Algoritmo 7 Recocido simulado

Entrada: grafo K, factor de enfriamiento $\alpha = 0{,}95$, temperatura mínima $T_{\text{mín}}$
Salida: ruta y su coste

```
ruta ← ruta al azar ; mejor ← ruta ; T ← 0,15 · C(ruta)
mientras T > T_mín hacer
    repetir n_iter veces:
        proponer invertir un tramo al azar (cambio 2-opt) y calcular ΔF
        si ΔF ≤ 0 o azar(0, 1) < e^(−ΔF/T) entonces aceptar el cambio
        actualizar mejor si la ruta mejora
    T ← α · T                                                  enfriar
fin mientras
devolver mejor
```

Complejidad y propiedades. Es una metaheurística estocástica: no garantiza el óptimo ni ofrece una cota de calidad, y el resultado depende del azar (por eso se mide con varias semillas). Evaluar el efecto de un cambio de 2-opt cuesta $O(1)$ (solo cambian cuatro aristas), pero aceptar el cambio obliga a actualizar la ruta en $O(n)$; el número de iteraciones por nivel de temperatura crece con n. Funciona con grafos dirigidos y no dirigidos. Su implementación es obra de Juan, que además calcula una cota inferior del óptimo con un 1-árbol para acotar el error de cada ejecución.

4.10 Algoritmo genético

Idea. Los algoritmos genéticos [15] imitan la evolución de las especies: se mantiene una *población* de rutas que evoluciona durante muchas *generaciones*; las rutas más cortas tienen más probabilidad de reproducirse, sus «hijos» combinan características de los padres y, de vez en cuando, sufren pequeñas *mutaciones*. Los elementos son:

- **Individuo:** una ruta, es decir, una permutación de las ciudades (la de partida va fija en primer lugar). Siempre es válida.
- **Aptitud:** el coste del ciclo; cuanto menor, mejor.
- **Selección por torneo:** cada padre es el mejor de 3 individuos elegidos al azar.
- **Cruce por orden (OX)** [6]: el hijo hereda de un padre un tramo seguido y el resto de ciudades en el orden en que aparecen en el otro padre, así nunca repite ni omite ninguna.
- **Mutación guiada:** con probabilidad p_m se prueban $q = 10$ inversiones de tramo al azar (el movimiento del 2-opt) y el individuo se queda con la mejor, *solo si mejora* su coste.
- **Elitismo:** las 2 mejores rutas pasan intactas a la siguiente generación.

Algoritmo 8 Algoritmo genético

Entrada: grafo K, población $P = 150$, generaciones máximas $G = 500$, probabilidad de mutación $p_m = 0{,}5$
Salida: ruta y su coste

```
poblacion ← P rutas aleatorias
repetir
    nueva ← las 2 mejores rutas de poblacion
    mientras |nueva| < P hacer
        p1, p2 ← dos padres por torneo (el mejor de 3 al azar)
        hijo ← CruceOX(p1, p2)
        con probabilidad pm, MutaciónGuiada(hijo)
        añadir hijo a nueva
    fin mientras
    poblacion ← nueva  ;  actualizar la mejor ruta
hasta que G generaciones o 150 sin mejorar
devolver mejor ruta
```

Complejidad y propiedades. Cada generación cruza y evalúa P rutas de longitud n (y la mutación prueba q variantes), así que el tiempo es $O(G \cdot P \cdot n \cdot q)$, *lineal en* n por generación, y la memoria $O(P \cdot n)$. Es estocástico, pero reproducible fijando la semilla, y no tiene garantía de calidad. Una lección del desarrollo merece señalarse: la primera versión, con mutaciones aleatorias simples, quedaba entre un 19 % y un 37 % por encima del óptimo con 47 ciudades; la mutación guiada (que incorpora una pequeña búsqueda local) lo redujo a un 2–5 %.

4.11 Quién hizo qué

Cuatro personas implementaron los ocho algoritmos. Pablo programó la fuerza bruta y el vecino más cercano, Jiahao programó DFJ, Juan programó el recocido simulado y Raimundo programó Held–Karp, 2-opt, Christofides y el genético. Cada algoritmo vive en su propio *notebook* con la misma estructura (ficha, teoría, código, pruebas y resultados), que una herramienta integra después en el *notebook* principal. El capítulo 9 describe este flujo de trabajo.

Capítulo 5

Metodología experimental

Este capítulo describe cómo se diseñó y se ejecutó el estudio empírico: qué se mide, sobre qué instancias, con qué reglas de aplicabilidad y con qué precauciones para que los resultados sean fiables y reproducibles.

5.1 Qué se mide

Para cada ejecución de un algoritmo sobre una instancia se registran cuatro magnitudes:

- el **coste** C de la ruta devuelta (en kilómetros o en horas, según la versión);
- el **tiempo** de cálculo, medido con `perf_counter`;
- el **pico de memoria** reservada por Python, medido con `tracemalloc`; y
- la **desviación respecto al óptimo**, definida para cada instancia como

$$\delta = \frac{C - C^*}{C^*} \times 100\,\%, \tag{5.1}$$

  donde C^* es el coste óptimo de esa instancia.

El óptimo C^* de cada instancia es el coste que devuelve **DFJ**, un algoritmo exacto que se ejecuta en todas las versiones. Para comprobar que esta referencia es fiable, el estudio verifica que *los tres algoritmos exactos coinciden* siempre que se pueden comparar (sección 5.6). Un caso aparte es la memoria de DFJ, de la que solo se cuenta la reservada por Python; la del *solver* CBC, que corre como proceso aparte, no se mide.

5.2 Instancias: dos estudios complementarios

El estudio se divide en dos partes que responden a preguntas distintas:

Estudio A: versiones reales.
Los **240 grafos de ciudades distintos** del catálogo (capítulo 3): 4 redes, 2 conjuntos de ciudades, 16 ámbitos geográficos, 2 pesos y dirigido o no dirigido. De ellos, 32 son de toda España (44 a 75 ciudades) y 208 son regionales (4 a 13 ciudades). Sirve para ver cómo cambian los resultados con la red, el peso y el sentido de las vías.

Estudio B: curva de escalado.
Subconjuntos **aleatorios** de las 75 ciudades, de $n = 5, 6, \ldots, 12, 14, 16, 18, 20, 25, 30, 40, 47, 60$ y 75 ciudades, con tres subconjuntos distintos por tamaño (uno solo para $n = 75$, que es el conjunto completo), sobre la versión `completo`/km, dirigida y no dirigida. Los subconjuntos se generan con semilla fija, de modo que son reproducibles. Sirve para ajustar cómo crecen el tiempo y la memoria con n.

5.3 Qué algoritmo se puede aplicar a qué versión

No todos los algoritmos se pueden ejecutar sobre todas las versiones, y esa limitación es un resultado en sí misma. Se aplican tres reglas, que el programa del estudio registra junto con su motivo (tabla 5.1):

- **Fuerza bruta:** solo con $n \leq 11$ ciudades. Hay $(n-1)!$ rutas: con 11 ciudades son 3,6 millones y tarda unos segundos; con 12, más de un minuto, y con 13, del orden de media hora.
- **Held–Karp:** solo con $n \leq 21$ ciudades, porque su tabla ocupa $8\,(n-1)\,2^{n-1}$ bytes: 170 MB con 21 ciudades y más de 700 MB con 22.
- **Christofides:** solo en grafos *no dirigidos*, porque necesita distancias simétricas.

Tabla 5.1. Aplicabilidad de cada algoritmo sobre los 240 grafos del estudio A.

Algoritmo	Versiones	Motivo de la exclusión
Fuerza bruta	192	más de 11 ciudades (las 32 de toda España y las 16 de Andalucía, con 13 ciudades)
Held–Karp	208	más de 21 ciudades (las 32 de toda España)
Christofides	120	necesita un grafo no dirigido: se excluyen las 120 versiones dirigidas
DFJ, vecino más cercano, 2-opt, recocido, genético	240	ninguna

Una consecuencia importante: en las 32 versiones de toda España (44 a 75 ciudades), **DFJ es el único algoritmo exacto aplicable**; los exactos de enumeración solo se pueden usar en las versiones regionales y en el estudio B hasta 11 y 20 ciudades.

5.4 Medir el tiempo: paralelo y en serie

El estudio completo supone miles de ejecuciones, que se reparten entre varios procesos. Pero el coste de una ruta no depende de la carga del ordenador y el tiempo sí: con varios procesos a la vez se comparten núcleos físicos, memoria caché y reloj. El equipo usado es un portátil con un procesador de cuatro núcleos físicos (ocho hilos) y frecuencia sostenida baja, y comprobamos que con seis procesos en paralelo la fuerza bruta de 11 ciudades pasa de unos 6 s a más de 40 s. Por eso el estudio se hace en **dos pasadas**:

1. **En paralelo** (seis procesos, 4.470 ejecuciones, unos 42 minutos): se usa para los *costes* y las comparaciones de calidad, que no dependen de la carga.
2. **En serie** (un solo proceso, 236 ejecuciones, unos 15 minutos): repite el estudio B con una instancia y una semilla por tamaño, con el equipo en reposo, y se usa para los *tiempos y la memoria*.

Una **calibración** repite en serie 15 tareas de la pasada en paralelo y mide cuánto las distorsiona (sección 6.7).

5.5 Algoritmos con azar y otros controles

- **Semillas.** El genético se ejecuta con 3 semillas y el recocido con 5; los resultados de estos dos algoritmos se resumen con medias y dispersión.
- **Ciudad de partida.** Es siempre Madrid (o la primera ciudad si Madrid no está en la instancia), para que las filas sean comparables. El vecino más cercano depende de ella; el resto no cambia el óptimo.
- **Ruido en tiempos pequeños.** Si una ejecución determinista dura menos de 50 ms, se repite cuatro veces más y se toma el mínimo.
- **Parada de seguridad.** Si pasan 15 minutos sin ningún resultado nuevo, el programa se detiene, y se puede reanudar sin perder lo hecho porque cada resultado se guarda al llegar.
- **Código real.** Los algoritmos no se copian en el programa del estudio: se extrae el código de sus *notebooks*, de modo que lo que se mide es exactamente lo que se explica en el trabajo.

5.6 Validación cruzada y control de calidad

Además de medir, el estudio contrasta los algoritmos entre sí. En cada instancia se comprueba que: (i) fuerza bruta, Held–Karp y DFJ devuelven el mismo coste (tolerancia del 0,1 %, la del *solver*); (ii) ninguna heurística devuelve un coste inferior al óptimo, lo que delataría un error; y (iii) cada ruta es válida (visita cada ciudad una vez) y su coste coincide con el recalculado.

Este control resultó decisivo: al ejecutar por primera vez los algoritmos sobre grafos *no dirigidos*, el recocido simulado entró en un bucle infinito y el modelo de DFJ se planteaba mal; ambos fallos se debían a cómo trataban las aristas de un grafo no dirigido y no se habían detectado en las pruebas individuales, hechas solo con grafos dirigidos. Se corrigieron con pocas líneas y se documentaron (sección 9.4).

5.7 Amenazas a la validez

- **Tamaño de las instancias.** El máximo son 75 ciudades; con muchas más, DFJ dejaría de ser inmediato y los metaheurísticos podrían ganar terreno. No se ha medido.
- **Un solo subconjunto con $n = 75$.** Es el conjunto completo; los tamaños menores usan tres subconjuntos distintos.
- **Parámetros por defecto.** Los metaheurísticos no se ajustaron a cada tamaño; con otros parámetros los resultados podrían cambiar.
- **Memoria incompleta en DFJ.** Solo se contabiliza la memoria reservada por Python, no la del *solver*.
- **Un único equipo.** Los tiempos absolutos dependen del ordenador; lo que importa son las proporciones y el crecimiento.

Capítulo 6

Resultados

Se presentan los resultados del estudio empírico (4.470 ejecuciones en paralelo y 236 en serie, sin ningún fallo), siguiendo las cinco preguntas del capítulo 1. Los tiempos absolutos dependen del equipo; interesan sobre todo las proporciones y el crecimiento con el número de ciudades.

6.1 Validación: los algoritmos exactos coinciden

Fuerza bruta, Held–Karp y DFJ dan exactamente el mismo coste en todas las instancias en las que se pueden comparar: **234 instancias** (fuerza bruta frente a DFJ) y **280** (Held–Karp frente a DFJ), con cero discrepancias. Ninguna heurística devuelve un coste inferior al óptimo. Este resultado respalda tanto la corrección de las tres implementaciones, que son muy distintas entre sí (enumeración, programación dinámica y programación entera), como el uso de DFJ como referencia del óptimo.

6.2 Complejidad: tiempo y memoria frente al tamaño

Las figuras 6.1 y 6.2 muestran, con el eje vertical logarítmico, cómo crecen el tiempo y la memoria con el número de ciudades (estudio B, grafo dirigido, mediana de las instancias; medido en serie). En escala logarítmica, una recta ascendente indica crecimiento exponencial y una curva que se aplana, crecimiento polinómico.

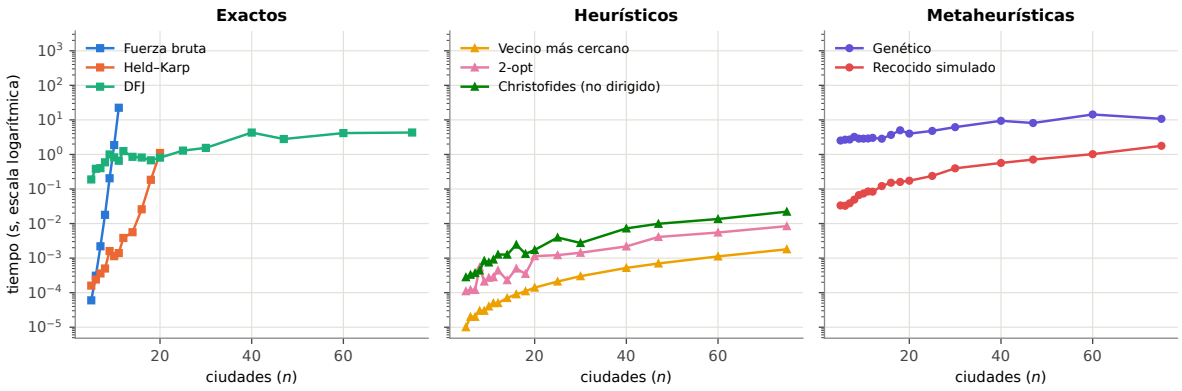


Figura 6.1. Tiempo de cálculo frente al número de ciudades (escala logarítmica en el eje vertical). La fuerza bruta y Held–Karp se detienen en sus límites (11 y 20 ciudades).

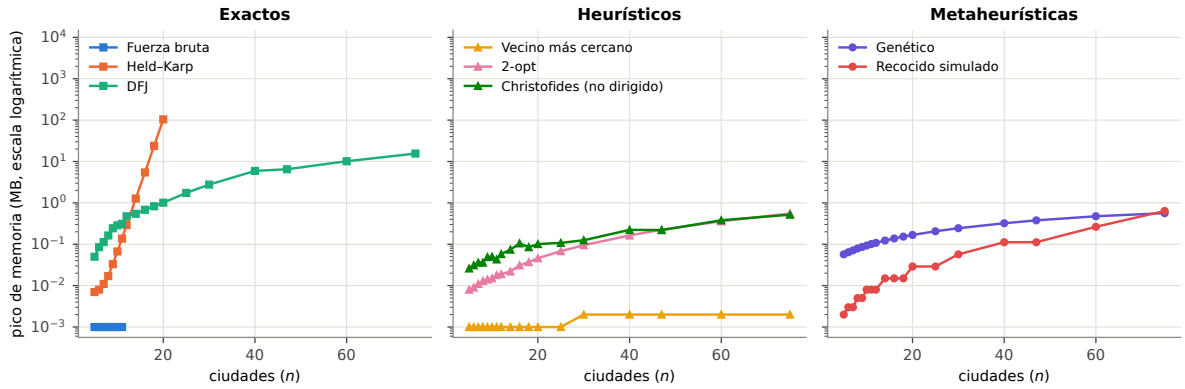


Figura 6.2. Pico de memoria reservada por Python frente al número de ciudades. La memoria de Held–Karp se duplica con cada ciudad.

6.2.1 Complejidad en notación O: teoría frente a observación

Para contrastar la complejidad teórica con la observada se ajustan los tiempos y la memoria medidos: un crecimiento exponencial $t \approx a\,b^n$ para los dos algoritmos de enumeración (donde b es el factor por ciudad añadida) y una ley de potencias $t \approx a\,n^k$ para los demás (ajuste lineal en escala logarítmica). El resultado se expresa en notación O en la tabla 6.1.

Tabla 6.1. Complejidad teórica y observada en notación O.

Algoritmo	Tiempo (teoría)	Tiempo (observado)	Memoria (teoría)	Memoria (observada)
Fuerza bruta	$O((n-1)!\cdot n)$	$O(10{,}6^n)$	$O(n)$	$\approx$ constante
Held–Karp	$O(n^2\cdot 2^n)$	$O(2{,}1^n)$	$O(n\cdot 2^n)$	$O(2{,}1^n)$
DFJ	$O(2^n)$ (peor caso)	$O(n)$	$O(n^2)$ en la práctica	$O(n^2)$
Vecino más cercano	$O(n^2)$	$O(n^{1{,}9})$	$O(n)$	$\approx$ constante
2-opt	$O(n^3)$	$O(n^{1{,}8})$	$O(n^2)$	$O(n^{1{,}8})$
Christofides	$O(n^3)$	$O(n^{1{,}6})$	$O(n^2)$	$O(n^{1{,}1})$
Recocido simulado	entre $O(n)$ y $O(n^2)$	$O(n^{1{,}5})$	$O(n^2)$	$O(n^{2{,}1})$
Genético	$O(G\cdot P\cdot n\cdot q)$	$O(n^{0{,}9})$	$O(P\cdot n)$	$O(n^{0{,}9})$

Los dos algoritmos exactos de enumeración crecen de forma **exponencial**, en consonancia con la teoría: la fuerza bruta multiplica su tiempo por 10,6 con cada ciudad añadida ($R^2 = 1{,}00$; la teoría dice "por n", es decir, entre 9 y 11 en este rango) y Held–Karp por 2,1 ($R^2 = 0{,}96$; la teoría dice aproximadamente 2), con una memoria que también se duplica por ciudad. El resto de algoritmos son **polinómicos**. El vecino más cercano ($n^{1{,}9}$) y el genético ($n^{0{,}9}$, lineal en n por generación) coinciden con lo esperado; el 2-opt y Christofides salen con exponentes menores que los teóricos, porque con 5 a 75 ciudades pesan los costes fijos y las operaciones vectorizadas de NumPy, y el exponente teórico se refiere al peor caso. DFJ es el caso más llamativo: su peor caso es exponencial, pero en estas instancias el tiempo crece apenas de forma lineal (de 0,2 s con 5 ciudades a 4,3 s con 75).

Conviene recordar que la O observada es una *estimación* con tamaños de 5 a 75 ciudades y no una demostración.

6.3 De enumerar a ser inteligente: la mejora entre algoritmos exactos

La tabla 6.2 compara el tiempo necesario para resolver al óptimo una instancia de n ciudades con los tres algoritmos exactos (serie, grafo dirigido).

Tabla 6.2. Tiempo (s) para resolver al óptimo una instancia de n ciudades. «–» indica que el algoritmo no es aplicable.

n	Fuerza bruta	Held–Karp	DFJ	FB / HK	DFJ / HK
8	0,018	0,0005	0,59	36	1.184
10	1,87	0,0011	0,82	1.641	747
11	22,5	0,0014	0,66	16.047	469
12	–	0,0038	1,25	–	328
14	–	0,0056	0,85	–	152
16	–	0,026	0,82	–	31,5
18	–	0,18	0,68	–	3,7
20	–	1,10	0,81	–	0,73
30	–	–	1,55	–	–
47	–	–	2,80	–	–
75	–	–	4,31	–	–

Held–Karp es **36 veces más rápido** que la fuerza bruta con 8 ciudades, **1.600 veces** con 10 y **16.000 veces** con 11. DFJ, en cambio, es *más lento* que Held–Karp con pocas ciudades (el arranque del *solver* cuesta entre 0,2 y 1 s: con 11 ciudades es 469 veces más lento), pero **no tiene su pared**: las curvas se cruzan hacia las **20 ciudades** (0,81 s de DFJ frente a 1,10 s de Held–Karp), y DFJ resuelve 75 ciudades al óptimo en 4,3 s. En resumen: de 5 a 7 ciudades cualquiera sirve; de 8 a 19, Held–Karp; **desde 20, DFJ**.

6.4 Calidad: cuánto se aleja del óptimo lo que no lo garantiza

La tabla 6.3 y la figura 6.3 muestran la desviación media respecto al óptimo de los algoritmos no exactos (estudio B; media de los subconjuntos y las semillas).

Tabla 6.3. Desviación media respecto al óptimo (%), por número de ciudades. Grafo dirigido, salvo Christofides (no dirigido).

n	Vecino más cercano	Christofides	2-opt	Recocido	Genético
11	8,0	7,6	0,4	0,0	0,0
20	20,9	5,3	1,1	0,1	1,4
47	30,4	14,6	1,1	2,3	5,0
75	31,8	13,6	4,5	4,4	6,1

- El **vecino más cercano** es siempre el peor (entre un 3 % y un 32 % por encima del óptimo de media, hasta un 42 % en un caso concreto) y empeora con el tamaño: solo sirve como punto de partida.

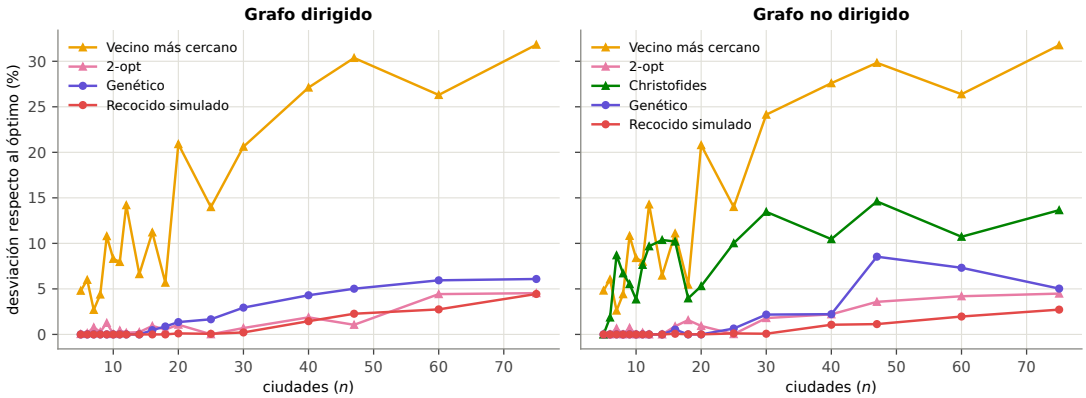


Figura 6.3. Desviación media respecto al óptimo (%) en función del número de ciudades, en el grafo dirigido (izquierda) y no dirigido (derecha).

- **Christofides** cumple siempre su garantía (peor caso individual medido: +19,7 %, muy por debajo del 50 %), pero queda por encima del 2-opt: la garantía es un peor caso, no una predicción.
- El **2-opt** es el mejor equilibrio: con un error del 1–5 % tarda milisegundos.
- **Recocido y genético llegan al óptimo hasta unas 14 ciudades** y después se alejan poco a poco. No mejoran al 2-opt en estos tamaños y tardan entre 170 y 2.000 veces más.

6.4.1 Variabilidad de los algoritmos con azar

Un solo resultado no describe a un algoritmo estocástico. Con 47 ciudades, el genético va del 1,0 % al 8,3 % de desviación (mediana 5,2 %) y el recocido del 0,0 % al 6,6 % (mediana 2,2 %); la figura 6.4 muestra cada ejecución. Las medias esconden además casos malos: en la peor versión de toda España, el 2-opt llega a +18 %, el recocido a +26 % y el genético a +49 %.

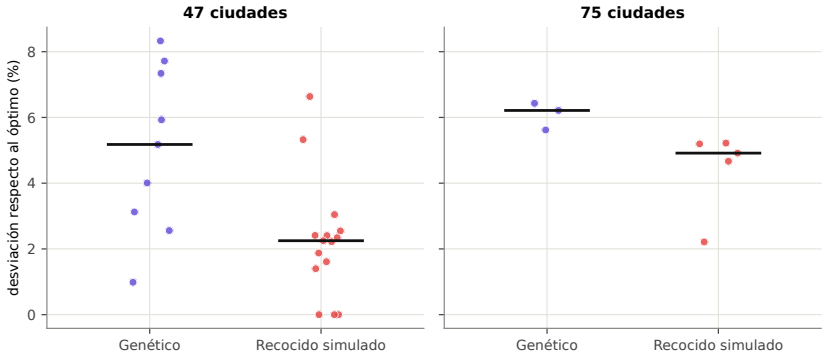


Figura 6.4. Desviación de cada ejecución del genético y del recocido simulado (grafo dirigido). La raya negra es la mediana.

6.5 Efecto de la versión del grafo

Los resultados dependen tanto de la versión como del algoritmo. La figura 6.5 muestra, para las 47 capitales, tres rutas sobre las carreteras reales.



Figura 6.5. Rutas de las 47 capitales (versión `completo`/km, con sentidos únicos, saliendo de Madrid): vecino más cercano, 2-opt y óptimo (DFJ), dibujadas sobre el trazado de las carreteras.

Red de carreteras. La tabla 6.4 da el óptimo en cada versión de toda España. Sin autovías el óptimo cuesta un **30–38 % más de kilómetros y casi el doble de tiempo** (+99 % a +108 %); con `autovias_principales`, solo un 3–4 % más. Con `solo_autovias` cuesta un 32–38 % más de kilómetros y quedan fuera tres ciudades.

Sentido de las vías. Ignorar los sentidos únicos apenas cambia el óptimo en la mayoría de las redes (0,7–4,0 %), pero en `solo_autovias` los sentidos cuestan un **13–19 %**, porque en una red solo de autovías no se puede dar la vuelta. Los heurísticos lo notan: con grafo dirigido el 2-opt se aleja de media un 6,9 % (2,6 % en no dirigido), el genético un 10,6 % (4,3 %) y el recocido un 5,2 % (2,3 %).

Calidad de los heurísticos según la red. La tabla 6.5 muestra la desviación media por red. `solo_autovias` es la más difícil para los heurísticos (2-opt 8,1 %, recocido 6,7 %, genético 13,4 %) y `sin_autovias`, la mejor para el 2-opt (1,8 %).

Tabla 6.4. Óptimo (DFJ) en las versiones de toda España y coste respecto a la red completa (47 y 75 ciudades; con 44 y 72 en `solo_autovias`).

		Óptimo en km		Óptimo en horas	
Red	Ciudades	dirigido	no dirigido	dirigido	no dirigido
`completo`	47	5.917,6	5.875,3	63,3	62,2
	75	6.146,2	6.069,8	67,7	65,9
`autovias_principales`	47	6.094,7	6.054,0	64,7	63,5
	75	6.377,5	6.242,6	69,8	67,1
`sin_autovias`	47	7.661,4	7.573,7	125,7	123,8
	75	8.465,7	8.331,4	140,6	137,4
`solo_autovias`	44 / 47	7.814,1	6.900,1	70,1	61,3
	72 / 75	8.509,3	7.222,6	76,5	64,6

Tabla 6.5. Desviación media (%) de cada algoritmo en las 32 versiones de toda España, según la red de carreteras.

Algoritmo	`completo`	`autovias_principales`	`sin_autovias`	`solo_autovias`
Vecino más cercano	19,7	26,3	30,8	23,4
Christofides	12,1	12,6	13,9	11,5
2-opt	4,6	4,6	1,8	8,1
Recocido simulado	2,9	2,7	2,7	6,7
Genético	4,7	5,5	6,3	13,4

Kilómetros frente a tiempo. La ruta más corta no es la más rápida. Para las 47 capitales, la ruta de mínimos kilómetros mide 5.917,6 km y tarda 64,5 h; la de mínimo tiempo mide 6.002,4 km y tarda 63,3 h. La ruta más rápida ahorra **1,2 h (1,9 %)** a cambio de **84,8 km (+1,4 %)**, y las dos rutas comparten 31 de sus 47 tramos entre ciudades.

6.6 Calidad frente a tiempo: ¿qué conviene usar?

La figura 6.6 sitúa cada algoritmo según su tiempo (eje horizontal, logarítmico) y su desviación media (eje vertical): lo mejor está abajo y a la izquierda. Con 11 ciudades se comparan todos; con 75 los exactos de enumeración ya no caben.

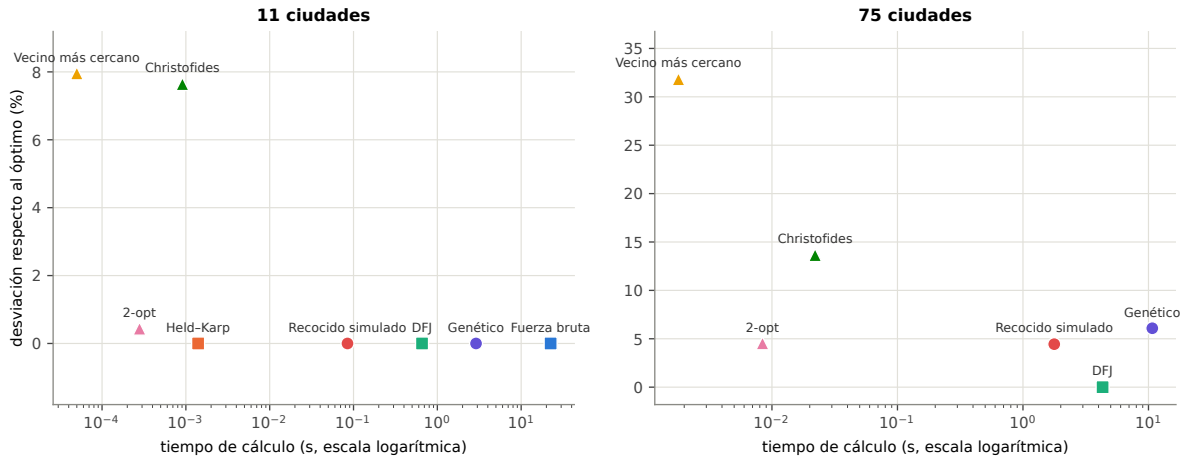


Figura 6.6. Calidad frente a tiempo con 11 y con 75 ciudades (grafo dirigido; Christofides, no dirigido).

Tabla 6.6. Resumen con 47 y 75 ciudades (grafo dirigido): calidad, tiempo (serie) y memoria.

n	Algoritmo	Desv. media (%)	Peor (%)	Tiempo (s)	Memoria (MB)
47	DFJ	0,0	0,0	2,795	6,51
	2-opt	1,1	1,6	0,004	0,22
	Recocido simulado	2,3	6,6	0,710	0,11
	Genético	5,0	8,3	8,124	0,38
	Vecino más cercano	30,4	36,1	0,001	$<$ 0,01
75	DFJ	0,0	0,0	4,310	15,54
	Recocido simulado	4,4	5,2	1,775	0,64
	2-opt	4,5	4,5	0,008	0,56
	Genético	6,1	6,4	10,724	0,56
	Vecino más cercano	31,8	31,8	0,002	$<$ 0,01

6.7 Cuánto distorsiona medir en paralelo

La calibración repitió en serie 15 tareas de la pasada en paralelo. El tiempo en paralelo resultó **2,46 veces mayor** de mediana, entre ×2,06 (genético) y ×2,72 (recocido simulado). Esto confirma que los tiempos solo son fiables en la pasada en serie, y justifica haber medido los tiempos y la memoria de las curvas de complejidad en esa pasada.

Capítulo 7

Conclusiones

Este capítulo responde a las preguntas planteadas en la introducción, resume lo aprendido y recoge las limitaciones y el trabajo futuro.

7.1 Respuestas a las preguntas del estudio

P1. ¿Qué algoritmo se puede usar con qué versión? Los tres algoritmos exactos y el resto de métodos no son intercambiables. La fuerza bruta solo sirve con 11 ciudades o menos y Held–Karp con 21 o menos, por lo que solo se pueden aplicar a las versiones regionales (4 a 13 ciudades) y a subconjuntos pequeños; Christofides exige un grafo no dirigido. En las versiones de toda España (44 a 75 ciudades), el único exacto aplicable es DFJ. Los otros cinco algoritmos funcionan en las 240 versiones.

P2. ¿Cómo crecen el tiempo y la memoria? Los exactos de enumeración crecen de forma exponencial, como predice la teoría: la fuerza bruta multiplica su tiempo por 10,6 con cada ciudad y se agota en 11 (22 s; con 13 serían unos 50 minutos); Held–Karp lo multiplica por 2,1 y su memoria $O(n \cdot 2^n)$ es el verdadero límite (104 MB con 20 ciudades, inviable hacia las 21–22). Los demás algoritmos crecen de forma polinómica, con exponentes iguales o menores que los teóricos. DFJ, con un peor caso exponencial, resolvió las 75 ciudades al óptimo en 4,3 s.

P3. ¿Cuánto se pierde al renunciar al óptimo? Con 75 ciudades, el vecino más cercano queda un 32 % por encima del óptimo, Christofides un 14 %, el 2-opt un 4,5 %, el recocido un 4,4 % y el genético un 6,1 %. El 2-opt es el mejor equilibrio entre calidad y tiempo: con un 1–5 % de error tarda milisegundos. Los metaheurísticos llegan al óptimo con hasta unas 14 ciudades, pero con más no mejoran al 2-opt y tardan entre 170 y 2.000 veces más, además de variar entre ejecuciones.

P4. ¿Qué efecto tiene la versión del grafo? Tanto como el algoritmo. Sin autovías, el óptimo cuesta un 30–38 % más de kilómetros y casi el doble de tiempo. Con solo autovías, los sentidos únicos cuestan un 13–19 %, porque no se puede dar la vuelta; en las demás redes ignorarlos apenas cambia el óptimo (0,7–4 %). Los heurísticos son menos robustos en grafos dirigidos y en la red más dispersa. Y la ruta más corta no es la más rápida: la de mínimo tiempo ahorra 1,2 h (1,9 %) a cambio de 85 km más (+1,4 %).

P5. ¿Qué conviene usar? Depende del caso (tabla 7.1).

Tabla 7.1. Qué algoritmo usar en cada caso.

Si se quiere…	Usar	Notación O y comportamiento
El óptimo con hasta unas 19 ciudades	Held–Karp	$O(n^2 \cdot 2^n)$ en tiempo y $O(n \cdot 2^n)$ en memoria: muy rápido, con un muro de memoria
El óptimo con 20 o más ciudades	DFJ	exponencial en el peor caso, pero 4,3 s con 75 ciudades
Una buena ruta en milisegundos	2-opt	$O(n^2)$ por vuelta; 1–5 % de error
Una calidad garantizada	Christofides	$O(n^3)$; $\leq$ 1,5× el óptimo (solo grafo no dirigido)
Explorar con azar	Recocido o genético	casi lineales en n, con un coste fijo alto y variabilidad entre ejecuciones
Una ruta de partida	Vecino más cercano	$O(n^2)$; instantáneo, pero con un 3–32 % de error

7.2 Otras conclusiones

- **La garantía de Christofides es un peor caso, no una predicción.** Nunca superó el +20 %, muy por debajo del +50 % garantizado, pero queda por encima del 2-opt, que no ofrece ninguna garantía. Su valor está en la seguridad, no en la calidad media.
- **Una metaheurística bien ajustada importa.** La primera versión del genético, con mutaciones aleatorias simples, se quedaba entre un 19 % y un 37 % por encima del óptimo; añadir una pequeña búsqueda local en la mutación lo bajó al 2–5 %.
- **Lo que se mide depende de cómo se mide.** En paralelo, los tiempos salen unas 2,5 veces más lentos; por eso los costes salen de una pasada y los tiempos de otra. Y contrastar tres algoritmos exactos independientes dio cero discrepancias y descubrió dos fallos que las pruebas por separado no habían encontrado.
- **Un buen conjunto de datos es una parte del resultado.** Obtener un grafo vial fiable y justificar las versiones del problema ocupó una parte considerable del trabajo y condiciona todo lo demás.

7.3 Limitaciones

Los metaheurísticos se midieron con sus parámetros por defecto y pocas semillas (3 y 5); con 75 ciudades solo hay un subconjunto (el conjunto completo); la memoria de DFJ no cuenta la del *solver* CBC; y los tamaños llegan a 75 ciudades, donde DFJ sigue siendo inmediato. Con muchas más ciudades es probable que DFJ deje de serlo y los metaheurísticos pasen a ser la mejor opción, pero no se ha medido. Además, las distancias son de coche y no consideran tráfico, peajes ni obras, y las coordenadas de las ciudades proceden de OpenStreetMap y no de una fuente oficial.

7.4 Trabajo futuro

- Ajustar los parámetros del recocido y del genético a cada tamaño, y probar variantes más potentes (por ejemplo, búsqueda local de tipo Or-opt o Lin–Kernighan).
- Probar instancias de cientos o miles de ciudades, donde DFJ deja de ser inmediato, y comparar con soluciones de referencia como Concorde.
- Realizar el estudio de escala por comunidades autónomas con las versiones regionales, y ampliar el conjunto de ciudades a municipios más pequeños.
- Añadir restricciones realistas: ventanas horarias, peajes, tráfico o límites de jornada, y pasar de un único viajante a varios vehículos.
- Ampliar la web con un panel de resultados del estudio y un recorrido guiado del proyecto.

Capítulo 8

La web interactiva

8.1 Propósito

Los algoritmos del viajante son más fáciles de entender viéndolos funcionar que leyendo su pseudocódigo. Por eso, además del estudio, se ha construido una **web interactiva** que anima los ocho algoritmos sobre el mapa de España, con la ruta construyéndose paso a paso sobre las carreteras reales. Sirve como apoyo visual para la presentación del trabajo y puede quedar publicada de forma permanente.

8.2 Qué ofrece

La web tiene tres vistas:

Laboratorio.
Se elige un algoritmo, un conjunto de ciudades (pulsando sobre el mapa o la lista, o con ejemplos predefinidos de 6, 9, 13, 47 y 75 ciudades), la medida (kilómetros o tiempo) y si se respetan los sentidos únicos. Un reproductor permite ejecutar, pausar, avanzar y retroceder paso a paso, saltar al resultado final y cambiar la velocidad. Junto al mapa aparecen el **pseudocódigo con la línea actual resaltada**, una **explicación en lenguaje llano** de lo que ocurre en cada paso, el coste de la ruta actual y de la mejor encontrada, una gráfica de la evolución del coste y la ficha del algoritmo.

Carrera.
Varios algoritmos resuelven *a la vez* las mismas ciudades. Cada uno avanza durante el mismo tiempo de cálculo, de modo que el que termina antes es de verdad el más rápido. Al final aparece una tabla con el coste, la desviación frente al mejor y el tiempo de cálculo. Los algoritmos no aplicables se descartan con su motivo, igual que en el estudio.

Acerca de.
Explica qué es la web y cómo se usa.

Hay tema oscuro (mapa nocturno, pensado para proyector) y claro, botón de pantalla completa, zoom con la rueda del ratón y atajos de teclado.



Figura 8.1. Laboratorio: DFJ con las 47 capitales. La primera solución del problema de asignación se parte en varios circuitos sueltos (cada uno de un color) sobre las carreteras reales; el pseudocódigo resalta el paso actual.

8.3 Cómo está hecha

- **Sin servidor.** Todo se calcula en el navegador, con JavaScript. Cada algoritmo está escrito como un *generador* en el que cada paso emite un «fotograma» (ruta, aristas resaltadas, línea de pseudocódigo, explicación y coste), lo que permite reproducir, pausar y retroceder con el mismo código.
- **Mapa.** Un mapa esquemático propio (contorno de la península y países vecinos, y red de autovías y vías rápidas de fondo) dibujado con *canvas* y SVG, con zoom y desplazamiento.
- **Carreteras reales.** Para cada par de las 75 ciudades se ha precalculado, con el grafo del trabajo, el camino mínimo y se ha guardado su trazado simplificado y codificado (1,1 MB para 2.775 pares); las rutas se dibujan siguiendo esas carreteras.
- **Solucionadores.** DFJ usa *HiGHS* [16], un *solver* de programación entera compilado a WebAssembly, y Christofides usa el emparejamiento de Edmonds; ambas librerías (licencia MIT) se incluyen en el repositorio, de modo que la web funciona sin internet.
- **Datos.** Las distancias son las mismas tablas del trabajo (capítulo 3).



Figura 8.2. Laboratorio: ruta óptima de las 47 capitales (5.917,6 km) encontrada por DFJ tras cuatro iteraciones.

8.4 Cómo sabemos que es correcta

Los algoritmos de JavaScript se contrastan automáticamente con los resultados de Python: fuerza bruta, Held–Karp y DFJ dan el mismo óptimo (5.917,562 km con las 47 capitales, 460,438 km en Galicia, 1.251,307 km en Andalucía y 6.146,165 km con las 75 ciudades); el vecino más cercano y el 2-opt dan los mismos costes que en Python (6.772,1 km y 6.326,0 km con las 47 capitales); y Christofides, el genético y el recocido devuelven rutas válidas con costes dentro de lo esperado. Además se probaron las vistas con un navegador automatizado, sin errores de consola.

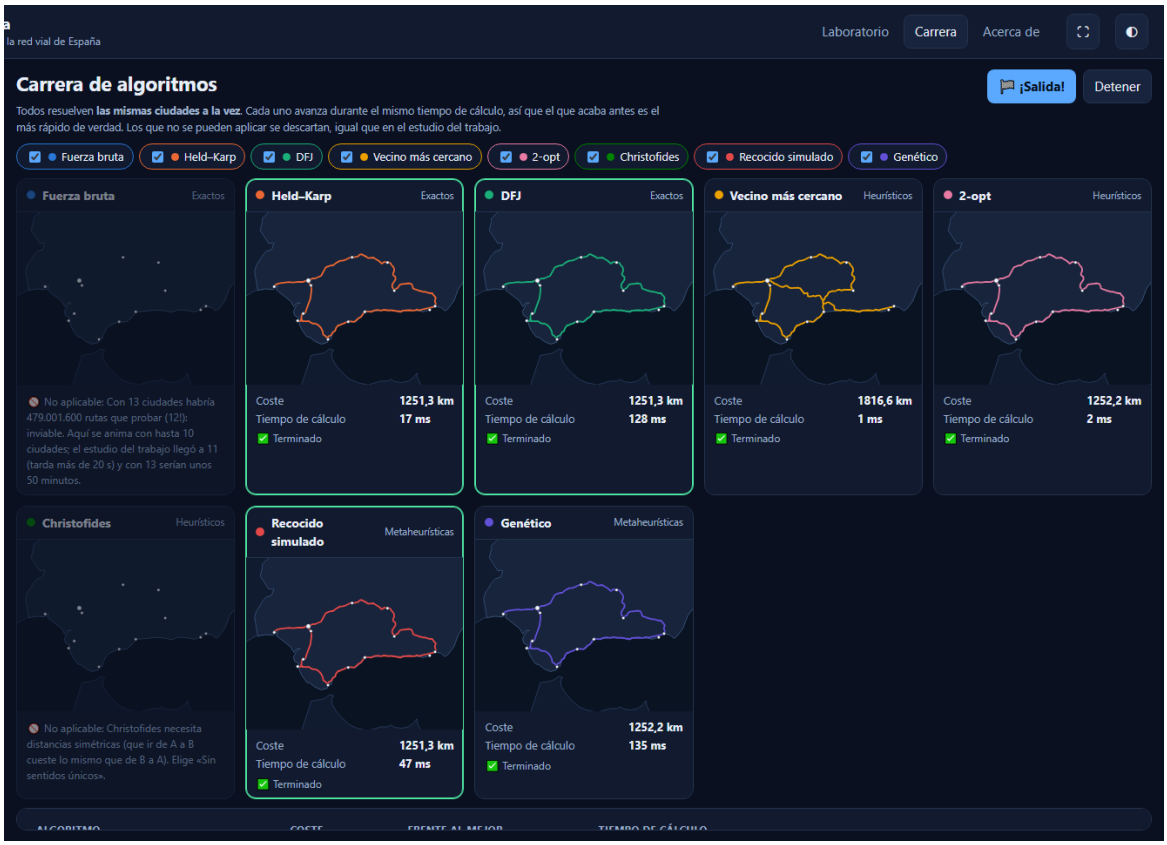


Figura 8.3. Carrera de algoritmos sobre las 13 ciudades de Andalucía. Fuerza bruta y Christofides se descartan con su motivo; Held–Karp, DFJ y el recocido encuentran el óptimo (1.251,3 km) en 17, 128 y 47 ms.

8.5 Publicación

La web es una carpeta del repositorio (`web/`) que se abre en local con un servidor sencillo (doble clic en `iniciar_web.bat` o `python -m http.server`). El repositorio incluye además una acción de GitHub que la publica en *GitHub Pages* cuando el propietario del repositorio la activa, de modo que queda en una dirección permanente. La guía de uso (documento aparte) explica el manejo paso a paso.

Capítulo 9

Organización del trabajo, uso de IA y lecciones aprendidas

9.1 Reparto del trabajo

El trabajo lo realizan cuatro personas (tabla 9.1). La construcción del grafo, las versiones, las herramientas de integración, el estudio empírico, la web y esta memoria las llevó Raimundo, que también coordinó al resto; los demás aportaron un algoritmo cada uno como mínimo (Raimundo, cuatro). El reparto de algoritmos se acordó en el grupo, y a cada persona se le asignó trabajar en su propia rama del repositorio.

Tabla 9.1. Autoría de cada parte del trabajo.

Persona	Contribución
Raimundo	Partes 1, 2, 4 y 5; Held–Karp, 2-opt, Christofides y genético; herramientas de integración; Anexo C; web; memoria y guía de uso; coordinación.
Pablo	Fuerza bruta y vecino más cercano.
Jiahao	DFJ (programación entera con restricciones de subciclo).
Juan	Recocido simulado (MCMC).

9.2 Flujo de trabajo con GitHub

El código se versiona en un repositorio de GitHub. Cada persona trabaja en su rama (`dev-pablo`, `dev_jiahao`, `dev_juan`, `dev-raimundo`) y entrega mediante *Pull Request* hacia la rama principal. Para evitar conflictos de git con un solo *notebook* compartido, cada algoritmo se programa en su **propio *notebook*** dentro de `parte3/`, a partir de una plantilla que ya contiene el código de preparación (la carga de las versiones del grafo). Una herramienta (`herramientas.py`) hace tres cosas: regenerar la plantilla a partir del *notebook* principal, *unificar* el formato de todos los *notebooks* (ficha, kernel, cabecera de pruebas) e *integrar* automáticamente los algoritmos en el *notebook* principal, de forma repetible. Todos los *notebooks* siguen la misma estructura: título, ficha (autor, tipo, complejidad, límite práctico), teoría, código, pruebas y resultados. Los datos de las fichas están en un único archivo (`fichas.json`), que alimenta también la tabla comparativa del *notebook* y las etiquetas de la web. Una guía (`CONTRIBUTING.md`) explica el proceso a los compañeros.

9.3 Uso de herramientas de inteligencia artificial

En el trabajo se ha usado **Claude Code** (Anthropic), un asistente de programación con modelos de lenguaje, para algunas partes. Para que el uso sea transparente, se registra qué se hizo con él (tabla 9.2). En todos los casos, el código y los resultados se ejecutaron y comprobaron antes de incorporarlos, y las pruebas de contraste entre algoritmos (capítulo 6) refuerzan esa comprobación.

Tabla 9.2. Uso de IA en el trabajo.

Parte	Herramienta	Para qué
Partes 1, 2 y 4; herramientas de integración; Anexo C; guía del equipo	Claude Code	Escribir y probar los *scripts* de descarga y construcción del grafo, el código del *notebook*, la plantilla, el estudio y su análisis.
Held–Karp, 2-opt, Christofides y genético	Claude Code	Teoría, código, pruebas y redacción de resultados; contrastados con la fuerza bruta y con DFJ.
Web interactiva y esta memoria	Claude Code	Programación de la web, generación de figuras y redacción, a partir de los datos y resultados del estudio.
Fuerza bruta, vecino más cercano (Pablo); DFJ (Jiahao); recocido simulado (Juan)	Cada autor debe indicar en `REGISTRO.md` y en su ficha el uso de IA que haya hecho.	

9.4 Problemas encontrados y lecciones aprendidas

Un trabajo de este tamaño no sale a la primera. Estos son los problemas más relevantes, cómo se resolvieron y qué enseñan:

- **La API Overpass no escalaba.** Solo se completó el 6 % de la descarga en once horas. Lección: para datos masivos, descargar un extracto y filtrarlo en local; ganó reproducibilidad (sección 2.3).
- **Falta de memoria al construir el grafo.** El flujo estándar necesitaba unos 12 GB. Lección: reescribir la simplificación con *arrays* (2,2 GB de pico), comprobando contra el grafo anterior que el resultado era idéntico.
- **Ejecuciones largas interrumpidas por falta de memoria.** Dos veces el sistema detuvo ejecuciones del *notebook* completo por memoria. Lección: dejar el equipo libre, y diseñar el estudio para que guarde cada resultado al llegar y se pueda reanudar.
- **Un algoritmo exacto dependía de la versión de una librería** (la de programación entera, PuLP, devuelve cosas distintas según la versión). Se escribió de forma compatible y se añadió la dependencia a los requisitos.
- **Dos fallos con grafos no dirigidos** (recocido simulado y DFJ) que solo apareció al ejecutar sistemáticamente todos los algoritmos sobre todas las versiones. Lección: probar cada algoritmo sobre *todos* los tipos de entrada que admite su contrato; el estudio sirvió también de prueba.
- **El paralelismo distorsiona los tiempos** unas 2,5 veces en el equipo usado. Lección: separar lo que no depende de la carga (costes) de lo que sí (tiempos), medidos en dos pasadas.
- **Cifras que no coincidían con lo medido.** En algún *notebook* un texto afirmaba tiempos o errores distintos de los que salían al ejecutarlo; se sustituyeron por las cifras medidas. Lección: generar las tablas de resultados a partir de la ejecución, no a mano.

9.5 Reproducibilidad

Todo se puede rehacer con los comandos del apéndice A: descargar el extracto y construir el grafo (unos 11 minutos), recalcular las tablas de distancias (unos 2 minutos), ejecutar el estudio completo (unos 60 minutos) y regenerar las figuras. El grafo grande no se guarda en el repositorio (supera el límite de 100 MB de GitHub), pero las tablas de distancias, los resultados del estudio y los datos de la web sí, de modo que quien programa un algoritmo o usa la web no necesita el grafo.

Apéndice A

Reproducir el trabajo

Desde la carpeta del proyecto y con las dependencias instaladas (`pip install -r requirements.txt`):

```
# 1. Descargar el extracto de OpenStreetMap y construir el grafo (anexos A y B)
python anexos/descargar_red.py          # ~1,4 GB, ~7 min, reanudable
python anexos/construir_grafo.py        # ~4 min, ~2,3 GB de RAM

# 2. Abrir el notebook (el trabajo): partes 1 a 5
jupyter lab trabajo_tsp.ipynb           # las tablas de distancias ya estan en data/
    versiones/

# 3. Repetir el estudio empirico (anexo C)
python anexos/ejecutar_estudio.py       # en paralelo, ~42 min
python anexos/ejecutar_estudio.py --serie   # tiempos y memoria, ~15 min (equipo en
    reposo)

# 4. Regenerar las figuras y compilar esta memoria
python memoria/figuras/generar_figuras.py
cd memoria && tectonic -X compile memoria.tex

# 5. Abrir la web
web/iniciar_web.bat                     # o: python -m http.server 8000 (dentro de web/)
```

Si solo se van a programar o probar algoritmos, no hace falta el grafo grande ni los anexos: las versiones ya están calculadas en `data/versiones/`, y basta con abrir el *notebook*, saltar la Parte 1 y ejecutar desde la Parte 2.

Apéndice B

Estructura del repositorio

Tabla B.1. Contenido del repositorio.

Ruta	Contenido
`trabajo_tsp.ipynb`	El trabajo: *notebook* principal con las cinco partes.
`anexos/`	Anexo A (descarga), Anexo B (construcción del grafo) y Anexo C (estudio empírico).
`parte3/`	Un *notebook* por algoritmo, plantilla, `fichas.json` y `herramientas.py`.
`data/ciudades.csv`	Las 75 ciudades (nombre, coordenadas, población, comunidad).
`data/versiones/`	16 tablas de distancias y catálogo de las 480 versiones.
`data/resultados/`	Resultados del estudio empírico (archivos CSV).
`web/`	La web interactiva (código, datos, librerías, pruebas y capturas).
`memoria/`	Esta memoria (fuente LaTeX y figuras) y su generador de figuras.
`guia/`	La guía de uso (fuente LaTeX).
`README.md`, `REGISTRO.md`, `CONTRIBUTING.md`	Documentación del proyecto, registro de autoría y uso de IA, y guía para colaborar.

Apéndice C

Fuentes y licencias

- **Carreteras:** © colaboradores de OpenStreetMap, bajo licencia ODbL; extracto de España de Geofabrik (`spain-latest.osm.pbf`, 19 de octubre de 2026).
- **Ciudades y población:** INE, *Cifras oficiales de población de los municipios españoles: Revisión del Padrón Municipal*, a 1 de enero de 2025 (BOE-A-2025-25362). Las coordenadas proceden de nodos de OpenStreetMap.
- **Contornos del mapa de la web:** Natural Earth (dominio público), a través del paquete `world-atlas`.
- **Librerías de la web:** HiGHS (`highs-js`) y `edmonds-blossom`, ambas con licencia MIT, incluidas en `web/vendor/`.

Bibliografía

[1] D. L. Applegate, R. E. Bixby, V. Chvátal y W. J. Cook. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.

[2] R. Bellman. Dynamic programming treatment of the travelling salesman problem. *Journal of the ACM*, 9(1):61–63, 1962.

[3] G. Boeing. OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65:126–139, 2017.

[4] N. Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem. Informe técnico 388, Graduate School of Industrial Administration, Carnegie Mellon University, 1976.

[5] G. A. Croes. A method for solving traveling-salesman problems. *Operations Research*, 6(6):791–812, 1958.

[6] L. Davis. Applying adaptive algorithms to epistatic domains. En *Proceedings of the 9th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 162–164, 1985.

[7] G. Dantzig, R. Fulkerson y S. Johnson. Solution of a large-scale traveling-salesman problem. *Operations Research*, 2(4):393–410, 1954.

[8] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

[9] J. Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965.

[10] M. R. Garey y D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[11] A. A. Hagberg, D. A. Schult y P. J. Swart. Exploring network structure, dynamics, and function using NetworkX. En *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pp. 11–15, 2008.

[12] M. Haklay y P. Weber. OpenStreetMap: User-generated street maps. *IEEE Pervasive Computing*, 7(4):12–18, 2008.

[13] C. R. Harris et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[14] M. Held y R. M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962.

[15] J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.

[16] Q. Huangfu y J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10:119–142, 2018.

[17] R. M. Karp. Reducibility among combinatorial problems. En R. E. Miller y J. W. Thatcher (eds.), *Complexity of Computer Computations*, pp. 85–103. Plenum Press, 1972.

[18] S. Kirkpatrick, C. D. Gelatt y M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[19] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller y E. Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.

[20] P. Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.